



Graduation project report cassava pest identification based on image processing and deep learning on python

kiến trúc máy tính (Trường Đại học Kiến trúc Hà Nội)



Scan to open on Studeersnel

TRƯỜNG ĐẠI HỌC KIẾN TRÚC HÀ NỘI

KHOA CÔNG NGHỆ THÔNG TIN

----- o0o -----



ĐỒ ÁN TỐT NGHIỆP KỸ SƯ

NGÀNH: CÔNG NGHỆ THÔNG TIN

MÃ NGÀNH: 7480201

KHÓA: 2020 - 2025

**ĐỀ TÀI: NHẬN DẠNG SÂU BỆNH CÂY SẴN DỰA
TRÊN XỬ LÝ ẢNH VÀ HỌC SÂU TRÊN PYTHON**

GIẢNG VIÊN HƯỚNG DẪN TS. BÙI HẢI PHONG

SINH VIÊN THỰC HIỆN TRẦN ĐĂNG QUYẾT

LỚP 20CN2

HÀ NỘI, 12 - 2024

TRƯỜNG ĐẠI HỌC KIẾN TRÚC HÀ NỘI

KHOA CÔNG NGHỆ THÔNG TIN

----- o0o -----



ĐỒ ÁN TỐT NGHIỆP KỸ SƯ

NGÀNH: CÔNG NGHỆ THÔNG TIN

MÃ NGÀNH: 7480201

KHÓA: 2020 - 2025

**ĐỀ TÀI: NHẬN DẠNG SÂU BỆNH CÂY SẢN DỰA
TRÊN XỬ LÝ ẢNH VÀ HỌC SÂU TRÊN PYTHON**

GIẢNG VIÊN HƯỚNG DẪN TS. BÙI HẢI PHONG

SINH VIÊN THỰC HIỆN TRẦN ĐĂNG QUYẾT

LỚP 20CN2

HÀ NỘI, 12 - 2024

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới thầy TS. Bùi Hải Phong, người đã tận tình hướng dẫn, định hướng và hỗ trợ em trong suốt quá trình thực hiện đề tài nghiên cứu này. Những kiến thức quý báu, sự góp ý chi tiết và động viên kịp thời của thầy là kim chỉ nam giúp em hoàn thành tốt đồ án tốt nghiệp của mình.

Em cũng xin bày tỏ lòng biết ơn tới các thầy cô trong khoa Công Nghệ Thông Tin, trường Đại học Kiến Trúc Hà Nội đã truyền đạt cho em những kiến thức nền tảng vững chắc trong suốt quá trình học tập tại trường. Những kiến thức đó là cơ sở quan trọng để em có thể thực hiện đề tài nghiên cứu này.

Xin gửi lời cảm ơn tới Công ty Nextx đã hỗ trợ cung cấp thông tin, dữ liệu và những nhận xét chuyên môn để em có thể hoàn thiện nghiên cứu của mình.

Em xin chân thành cảm ơn gia đình, bạn bè đã luôn bên cạnh động viên, khích lệ và giúp đỡ em trong suốt thời gian qua. Sự ủng hộ của mọi người là nguồn động lực to lớn để em vượt qua những khó khăn, thử thách và hoàn thành tốt nhiệm vụ của mình.

Cuối cùng, em xin cảm ơn tất cả những người đã trực tiếp hoặc gián tiếp giúp đỡ em hoàn thành đồ án tốt nghiệp này.

Em xin trân trọng cảm ơn!

LỜI CAM ĐOAN CỦA TÁC GIẢ

LỜI NÓI ĐẦU

Cây sắn, hay còn gọi là khoai mì, là một trong những cây lương thực quan trọng, đóng vai trò thiết yếu trong việc đảm bảo an ninh lương thực và cung cấp nguồn thu nhập cho hàng triệu nông hộ tại Việt Nam. Tuy nhiên, sản xuất sắn đang phải đối mặt với nhiều thách thức, trong đó sâu bệnh hại là một trong những nguyên nhân chính gây giảm năng suất và chất lượng củ sắn. Việc phát hiện và chẩn đoán bệnh kịp thời đóng vai trò then chốt trong việc triển khai các biện pháp phòng trừ hiệu quả, góp phần bảo vệ mùa màng và nâng cao thu nhập cho người nông dân.

Trước thực trạng đó, cùng với sự phát triển mạnh mẽ của công nghệ thông tin, đặc biệt là trong lĩnh vực xử lý ảnh và học sâu, tôi đã lựa chọn đề tài "Nhận dạng sâu bệnh cây sắn dựa trên xử lý ảnh và học sâu" với mong muốn góp phần vào việc ứng dụng các tiến bộ khoa học kỹ thuật vào sản xuất nông nghiệp, cụ thể là trong lĩnh vực bảo vệ thực vật.

Đề tài tập trung nghiên cứu và xây dựng một hệ thống tự động có khả năng nhận dạng một số loại sâu bệnh hại chính trên cây sắn dựa trên hình ảnh thu thập được. Hệ thống sử dụng các kỹ thuật tiên tiến trong lĩnh vực xử lý ảnh, kết hợp với các mô hình học sâu mạnh mẽ như YOLOv8, ResNet, EfficientNet hay ViT để phát hiện và phân loại bệnh trên lá sắn.

Qua quá trình nghiên cứu, tôi hy vọng rằng kết quả của đề tài sẽ góp phần cung cấp một công cụ hữu ích, hỗ trợ người nông dân trong việc giám sát và phát hiện sớm sâu bệnh hại trên cây sắn, từ đó đưa ra các biện pháp phòng trừ kịp thời và hiệu quả. Đồng thời, đây cũng là tiền đề cho việc ứng dụng rộng rãi hơn nữa các kỹ thuật học sâu và xử lý ảnh vào trong lĩnh vực nông nghiệp, hướng tới một nền nông nghiệp thông minh và bền vững.

Trong quá trình thực hiện đề tài, tôi đã nhận được sự hướng dẫn tận tình của thầy TS. Bùi Hải Phong cùng với sự hỗ trợ, giúp đỡ của các thầy cô trong khoa Công Nghệ Thông Tin, bạn bè và gia đình. Tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất tới tất cả mọi người.

Mặc dù đã có nhiều cố gắng, nhưng do kiến thức và kinh nghiệm còn hạn chế, đề tài chắc chắn không tránh khỏi những thiếu sót. Tôi rất mong nhận được những ý kiến đóng góp quý báu của quý thầy cô và các bạn để đề tài được hoàn thiện hơn.

Tôi xin trân trọng cảm ơn!

MỤC LỤC

LỜI CẢM ƠN.....	
LỜI CAM ĐOAN CỦA TÁC GIẢ.....	
LỜI NÓI ĐẦU.....	
MỤC LỤC.....	
DANH MỤC CÁC BẢNG.....	
DANH MỤC CÁC HÌNH.....	
DANH MỤC TỪ VIẾT TẮT, THUẬT NGỮ.....	
TÓM TẮT NỘI DUNG.....	
CHƯƠNG 1: GIỚI THIỆU.....	
1.1. Đặt vấn đề.....	
1.2. Mục tiêu nghiên cứu.....	
1.2.1. Mục tiêu chung.....	
1.2.2. Mục tiêu cụ thể.....	
1.3. Phạm vi và giới hạn của đề tài.....	
1.3.1. Phạm vi nghiên cứu.....	
1.3.2. Giới hạn nghiên cứu.....	
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÁC NGHIÊN CỨU LIÊN QUAN.....	
2.1. Cơ sở lý thuyết.....	
2.1.1. Xử lý ảnh (Image Processing).....	
2.1.2. Học sâu (Deep Learning).....	
2.1.3. Các kỹ thuật tăng cường dữ liệu (Data Augmentation).....	
2.1.4. Các phương pháp nâng cao hiệu quả của mô hình.....	
2.1.5. Explainable AI (XAI) - Trực quan hóa kết quả AI.....	
2.2. Các nghiên cứu liên quan.....	
2.2.1. Tổng quan về ứng dụng học sâu trong nông nghiệp.....	
2.2.3. Các nghiên cứu về nhận dạng bệnh trên cây sắn.....	
2.3. Công cụ và môi trường phát triển.....	
2.3.1. Ngôn ngữ lập trình Python.....	
2.3.2. Thư viện PyTorch và Timm.....	
2.3.3. Thư viện OpenCV.....	
2.3.4. Thư viện Scikit-learn.....	
2.3.5. Nền tảng Google Colab.....	
2.3.6. Thư viện Matplotlib và Seaborn.....	
2.3.7. Công cụ gán nhãn dữ liệu: MakeSense.ai.....	
2.3.8. Môi trường phân cứng và nền tảng huấn luyện.....	
CHƯƠNG 3: PHƯƠNG PHÁP VÀ THIẾT KẾ HỆ THỐNG.....	
3.1. Phương pháp nghiên cứu.....	
3.2. Thu thập và xử lý dữ liệu.....	

3.2.1. Mô tả dataset.....	
3.2.2. Gán nhãn dữ liệu.....	
3.2.3. Phân tích dữ liệu (Data Analysis).....	
3.3. Thiết kế hệ thống.....	
3.3.1. Kiến trúc tổng thể.....	
3.3.2. Quy trình xử lý.....	
3.3.3. Giải thích các bước triển khai.....	
3.4. Mô hình sử dụng.....	
3.4.1. YOLOv8.....	
3.4.2. Các mô hình classification.....	
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ.....	
4.1. Quá trình huấn luyện và thử nghiệm.....	
4.1.1. Môi trường thử nghiệm.....	
4.1.3. Quá trình huấn luyện các mô hình phân loại.....	
4.2. So sánh mô hình.....	
4.3. Trực quan hóa với XAI.....	
4.3.1. Giới thiệu về Grad-CAM.....	
4.3.2. Ứng dụng Grad-CAM cho ResNet50.....	
4.3.3. Phân tích kết quả Grad-CAM.....	

DANH MỤC CÁC BẢNG

DANH MỤC CÁC HÌNH

DANH MỤC TỪ VIẾT TẮT, THUẬT NGỮ

TÓM TẮT NỘI DUNG

CHƯƠNG 1: GIỚI THIỆU

1.1. Đặt vấn đề

Cây sắn (*Manihot esculenta* Crantz/cassava), còn được gọi là khoai mì, là một loại cây lương thực nhiệt đới và cận nhiệt đới, thuộc họ Thầu dầu (*Euphorbiaceae*). Với khả năng thích nghi tốt với điều kiện khí hậu khắc nghiệt, kỹ thuật canh tác đơn giản và năng suất cao, cây sắn đóng vai trò quan trọng trong việc đảm bảo an ninh lương thực và tạo sinh kế cho hàng triệu người dân tại nhiều quốc gia trên thế giới, đặc biệt là các nước đang phát triển.

Cục Xuất nhập khẩu (Bộ Công Thương) dẫn số liệu thống kê của Tổng cục Hải quan cho biết, tháng 11/2023, Việt Nam xuất khẩu 268,91 nghìn tấn sắn và các sản phẩm từ sắn, trị giá 134,77 triệu USD. Cây sắn không chỉ cung cấp nguồn lương thực cho con người mà còn là nguyên liệu quan trọng cho ngành công nghiệp chế biến thức ăn chăn nuôi, sản xuất tinh bột, ethanol sinh học và nhiều sản phẩm khác.

Tuy nhiên, sản xuất sắn tại Việt Nam đang phải đối mặt với nhiều thách thức, trong đó sâu bệnh hại là một trong những yếu tố chính làm giảm năng suất và chất lượng củ sắn. Hàng năm, sâu bệnh hại gây thiệt hại trung bình từ 10-15% sản lượng sắn, thậm chí có những vùng bị mất trắng do dịch bệnh bùng phát (Cục Bảo vệ thực vật, 2020). Một số loại sâu bệnh hại chính trên cây sắn bao gồm: bệnh khảm lá sắn (*Cassava Mosaic Disease*), bệnh chổi rồng (*Cassava Witches' Broom Disease*), rệp sáp bột hồng (*Phenacoccus manihoti*), nhện đỏ (*Tetranychus urticae*), sâu đục thân (*Chilo partellus*),...

Việc phát hiện và chẩn đoán chính xác, kịp thời các loại sâu bệnh hại đóng vai trò vô cùng quan trọng trong việc triển khai các biện pháp phòng trừ hiệu quả, hạn chế sự lây lan của dịch bệnh, bảo vệ mùa màng và nâng cao thu nhập cho người nông dân. Hiện nay, việc chẩn đoán sâu bệnh hại cây sắn chủ yếu vẫn dựa vào phương pháp thủ công, tức là người nông dân phải trực tiếp quan sát và nhận biết các triệu chứng bệnh trên cây trồng. Phương pháp này tốn nhiều thời gian, công sức, đòi hỏi người nông dân phải có kinh nghiệm và kiến thức chuyên môn, đồng thời dễ xảy ra sai sót, nhầm lẫn, đặc biệt là khi các triệu chứng bệnh ở giai đoạn đầu hoặc không điển hình.

Trong bối cảnh đó, cùng với sự phát triển mạnh mẽ của cuộc cách mạng công nghiệp 4.0, việc ứng dụng các tiến bộ khoa học kỹ thuật, đặc biệt là công nghệ thông tin vào sản xuất nông nghiệp đang trở thành xu hướng tất yếu. Các lĩnh vực

như xử lý ảnh (Image Processing), thị giác máy tính (Computer Vision) và học sâu (Deep Learning) đã đạt được nhiều thành tựu vượt bậc và được ứng dụng rộng rãi trong nhiều lĩnh vực, bao gồm cả nông nghiệp.

Nhận thức được tầm quan trọng của vấn đề nêu trên, đồng thời mong muốn góp phần ứng dụng các tiến bộ khoa học kỹ thuật vào thực tiễn sản xuất nông nghiệp, tôi đã lựa chọn đề tài: "Nhận dạng sâu bệnh cây sắn dựa trên xử lý ảnh và học sâu" làm đề tài nghiên cứu cho đồ án tốt nghiệp của mình.

1.2. Mục tiêu nghiên cứu

1.2.1. Mục tiêu chung

Mục tiêu chung của đề tài là nghiên cứu và xây dựng một hệ thống tự động có khả năng nhận dạng một số loại sâu bệnh hại chính trên cây sắn dựa trên hình ảnh thu thập được, từ đó hỗ trợ người nông dân trong việc giám sát và phát hiện sớm sâu bệnh hại, góp phần nâng cao hiệu quả sản xuất sắn.

1.2.2. Mục tiêu cụ thể

- Xây dựng cơ sở dữ liệu: Thu thập và xây dựng bộ dữ liệu ảnh về một số loại sâu bệnh hại chính trên cây sắn tại Việt Nam (bao gồm cả dữ liệu từ nguồn Kaggle và ảnh tự chụp). Tiến hành gán nhãn và tiền xử lý dữ liệu để phục vụ cho việc huấn luyện mô hình.
- Nghiên cứu và triển khai các mô hình học sâu: Tìm hiểu và triển khai các mô hình học sâu tiên tiến trong lĩnh vực nhận dạng ảnh, bao gồm các mô hình phát hiện đối tượng (Object Detection) như YOLOv8 và các mô hình phân loại ảnh (Image Classification) như ResNet, EfficientNet, MobileNet, Vision Transformer (ViT).
- Huấn luyện và đánh giá mô hình: Huấn luyện các mô hình đã lựa chọn trên bộ dữ liệu đã xây dựng. Đánh giá hiệu quả của các mô hình dựa trên các độ đo phù hợp như: Accuracy, Precision, Recall, F1-score, mAP.
- Tối ưu hóa mô hình: Áp dụng các kỹ thuật tối ưu hóa (Hyperparameter Tuning, Transfer Learning, Regularization...) để nâng cao hiệu quả của mô hình.
- Xây dựng hệ thống demo: Thiết kế và xây dựng một hệ thống demo (dưới dạng ứng dụng web hoặc ứng dụng di động) cho phép người dùng tải lên

hình ảnh lá sắn và nhận được kết quả chẩn đoán về loại sâu bệnh hại (nếu có) và cách khắc phục.

- Đánh giá khả năng triển khai thực tế: Thử nghiệm hệ thống trên các ảnh chụp thực tế và đánh giá tốc độ, độ chính xác trong điều kiện thực tế.

1.3. Phạm vi và giới hạn của đề tài

1.3.1. Phạm vi nghiên cứu

- Đối tượng: Tập trung vào việc nhận dạng các loại sâu bệnh hại trên lá sắn.
- Loại sâu bệnh: Nghiên cứu tập trung vào một số loại sâu bệnh hại phổ biến và gây thiệt hại nghiêm trọng trên cây sắn tại Việt Nam, bao gồm:
 - Bệnh khảm lá sắn
 - Bệnh cháy lá
 - Bệnh thối rễ
 - Bệnh đốm lá
- Khu vực: Dữ liệu ảnh tự chụp được thu thập chủ yếu tại một số vùng trồng sắn chính tại Việt Nam .
- Phương pháp: Sử dụng các kỹ thuật xử lý ảnh và các mô hình học sâu để phát hiện và phân loại sâu bệnh hại.

1.3.2. Giới hạn nghiên cứu

- Dữ liệu: Bộ dữ liệu tập trung chủ yếu vào các loại sâu bệnh hại đã nêu ở trên. Số lượng ảnh tự chụp cho mỗi loại bệnh có thể còn hạn chế.
- Môi trường thử nghiệm: Mô hình được huấn luyện và đánh giá chủ yếu dựa trên ảnh tĩnh (ảnh chụp lá sắn). Việc triển khai trên video thời gian thực chưa được thực hiện trong phạm vi của đề tài này.
- Phần cứng: Việc huấn luyện các mô hình học sâu đòi hỏi cấu hình phần cứng tương đối mạnh (GPU). Trong quá trình thực hiện đề tài, có thể gặp phải một số hạn chế về khả năng tính toán của phần cứng.
- Độ chính xác: Mặc dù các mô hình học sâu cho kết quả rất tốt, tuy nhiên độ chính xác của hệ thống vẫn có thể bị ảnh hưởng bởi các yếu tố như chất lượng ảnh đầu vào, điều kiện ánh sáng, mức độ nghiêm trọng của bệnh,...

1.4. Cấu trúc báo cáo

Báo cáo đồ án tốt nghiệp được trình bày gồm 5 chương với nội dung chính như sau:

- Chương 1: Giới thiệu: Trình bày tổng quan về đề tài nghiên cứu, bao gồm lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi và giới hạn của đề tài, cấu trúc báo cáo.
- Chương 2: Cơ sở lý thuyết và các nghiên cứu liên quan: Tổng quan về các kiến thức nền tảng liên quan đến đề tài, bao gồm các khái niệm cơ bản về xử lý ảnh, học sâu, các mô hình CNN, các kỹ thuật Explainable AI (XAI). Đồng thời, chương này cũng trình bày và phân tích các nghiên cứu liên quan đến việc ứng dụng xử lý ảnh và học sâu trong lĩnh vực nông nghiệp, đặc biệt là trong việc nhận dạng sâu bệnh hại cây trồng.
- Chương 3: Phương pháp và thiết kế hệ thống: Trình bày chi tiết về phương pháp nghiên cứu được sử dụng trong đề tài, bao gồm quy trình thu thập và xử lý dữ liệu, các mô hình học sâu được lựa chọn (YOLOv8, ResNet, EfficientNet, MobileNet, ViT), các kỹ thuật tối ưu hóa mô hình. Đồng thời, chương này cũng mô tả chi tiết về thiết kế tổng thể của hệ thống, bao gồm các bước triển khai và các hyperparameter của mô hình.
- Chương 4: Kết quả và đánh giá: Trình bày các kết quả thực nghiệm, bao gồm quá trình huấn luyện và đánh giá các mô hình trên bộ dữ liệu đã xây dựng. So sánh hiệu quả của các mô hình dựa trên các độ đo phù hợp. Phân tích các trường hợp lỗi và đề xuất các giải pháp cải thiện. Trình bày kết quả demo hệ thống và đánh giá khả năng triển khai thực tế.
- Chương 5: Kết luận và hướng phát triển: Tổng kết các kết quả đạt được của đề tài, đánh giá những ưu điểm và hạn chế của hệ thống. Đề xuất các hướng phát triển tiếp theo của đề tài trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÁC NGHIÊN CỨU LIÊN QUAN

2.1. Cơ sở lý thuyết

2.1.1. Xử lý ảnh (Image Processing)

Xử lý ảnh là một lĩnh vực khoa học máy tính tập trung vào việc thao tác và phân tích hình ảnh kỹ thuật số. Mục tiêu của xử lý ảnh là cải thiện chất lượng hình ảnh, trích xuất thông tin hữu ích từ ảnh, hoặc biến đổi ảnh cho các mục đích khác nhau. Trong nghiên cứu này, các kỹ thuật xử lý ảnh được sử dụng chủ yếu trong giai đoạn tiền xử lý dữ liệu, nhằm chuẩn hóa dữ liệu đầu vào và nâng cao hiệu quả cho các mô hình học sâu.

2.1.1.1. Các khái niệm cơ bản

- **Điểm ảnh (Pixel):** Là phần tử nhỏ nhất của một hình ảnh kỹ thuật số. Mỗi điểm ảnh có một vị trí xác định và chứa thông tin về màu sắc tại vị trí đó.
- **Độ phân giải (Resolution):** Là số lượng điểm ảnh trên một đơn vị độ dài (ví dụ: pixels per inch - PPI). Độ phân giải càng cao thì hình ảnh càng chi tiết.
- **Không gian màu (Color Space):** Là mô hình toán học mô tả cách thức biểu diễn màu sắc. Một số không gian màu phổ biến bao gồm RGB (Red, Green, Blue), HSV (Hue, Saturation, Value), Grayscale (ảnh xám).
- **Biểu đồ Histogram:** Là đồ thị biểu diễn tần suất xuất hiện của các mức xám hoặc các màu sắc trong ảnh. Biểu đồ Histogram cung cấp thông tin về phân bố độ sáng, độ tương phản của ảnh.
- **Độ tương phản (Contrast):** Là sự khác biệt về độ sáng giữa các vùng sáng nhất và tối nhất trong ảnh.

2.1.1.2. Tiền xử lý ảnh

Tiền xử lý ảnh là bước quan trọng trong quy trình xử lý ảnh, nhằm chuẩn hóa dữ liệu đầu vào, loại bỏ nhiễu và nâng cao chất lượng ảnh. Một số kỹ thuật tiền xử lý ảnh thường được sử dụng bao gồm:

- **Chuyển đổi không gian màu (Color Space Conversion):** Chuyển đổi ảnh từ không gian màu này sang không gian màu khác. Ví dụ: chuyển từ RGB sang Grayscale để giảm chiều dữ liệu.
- **Thay đổi kích thước ảnh (Resizing):** Điều chỉnh kích thước ảnh về kích thước chuẩn để phù hợp với đầu vào của mô hình học sâu.

- Chuẩn hóa (Normalization): Điều chỉnh giá trị pixel về một phạm vi nhất định (ví dụ: 0-1) để giúp mô hình học sâu hội tụ nhanh hơn.
- Lọc nhiễu (Noise Reduction): Loại bỏ các điểm ảnh nhiễu trong ảnh, giúp cải thiện chất lượng ảnh. Các bộ lọc thường được sử dụng bao gồm: lọc trung bình (Mean Filter), lọc trung vị (Median Filter), lọc Gaussian (Gaussian Filter).
- Điều chỉnh độ sáng, độ tương phản (Brightness/Contrast Adjustment): Tăng hoặc giảm độ sáng, độ tương phản của ảnh để làm nổi bật các đặc trưng quan trọng.

2.1.1.3. Phân tích thống kê về ảnh:

- Phân bố độ sáng (Brightness Distribution): Sử dụng biểu đồ histogram để phân tích sự phân bố mức độ sáng của ảnh, giúp đánh giá xem ảnh có quá tối, quá sáng hay phân bố đều. Điều này quan trọng vì nó ảnh hưởng trực tiếp đến khả năng nhận diện của model. Ví dụ, ảnh quá tối có thể làm mất chi tiết, trong khi ảnh quá sáng có thể làm "cháy" chi tiết.
- Phân bố màu sắc (Color Distribution): Phân tích sự phân bố của các kênh màu (ví dụ: R, G, B trong không gian màu RGB) để hiểu rõ hơn về đặc trưng màu sắc của từng loại bệnh. Ví dụ, bệnh đốm lá có thể có màu nâu đặc trưng, trong khi bệnh khảm lá có thể có sự pha trộn giữa màu xanh và vàng. Việc phân tích này giúp lựa chọn các phương pháp tăng cường dữ liệu phù hợp và cải thiện khả năng phân biệt giữa các loại bệnh.
- Độ tương phản (Contrast Analysis): Đánh giá độ tương phản của ảnh để xác định sự khác biệt giữa các vùng sáng và tối. Độ tương phản thấp có thể làm cho các đặc trưng của bệnh khó nhận diện hơn. Các phương pháp như Histogram Equalization có thể được sử dụng để cải thiện độ tương phản của ảnh, làm nổi bật các chi tiết quan trọng.
- Phân tích không gian màu (Color Space Analysis): Chuyển đổi ảnh sang các không gian màu khác nhau (ví dụ: HSV, LAB) để phân tích các đặc trưng màu sắc trong các không gian màu này. Ví dụ, trong không gian màu HSV, kênh Hue có thể cung cấp thông tin hữu ích về màu sắc của các đốm bệnh, giúp phân biệt giữa các loại bệnh khác nhau.

2.1.2. Học sâu (Deep Learning)

Học sâu (Deep Learning) là một nhánh của học máy (Machine Learning) dựa trên việc sử dụng các mạng nơ-ron nhân tạo (Artificial Neural Networks) với nhiều lớp (layer) để học các biểu diễn phức tạp của dữ liệu. Khác với các phương pháp học máy truyền thống, học sâu có khả năng tự động trích xuất các đặc trưng từ dữ liệu thô mà không cần các kỹ sư phải thiết kế các đặc trưng thủ công.

2.1.2.1. Mạng nơ-ron tích chập (Convolutional Neural Networks - CNNs)

Mạng nơ-ron tích chập (CNNs) là một loại mạng nơ-ron sâu được thiết kế đặc biệt để xử lý dữ liệu dạng lưới, chẳng hạn như hình ảnh. CNNs sử dụng các lớp tích chập (convolutional layers) để tự động học các đặc trưng từ ảnh, các lớp pooling (pooling layers) để giảm chiều dữ liệu và các lớp fully connected (fully connected layers) để thực hiện phân loại.

Kiến trúc cơ bản của CNN:

- Lớp tích chập (Convolutional Layer): Là lớp quan trọng nhất trong CNN, thực hiện phép toán tích chập giữa bộ lọc (filter/kernel) và đầu vào để trích xuất các đặc trưng.
- Lớp kích hoạt (Activation Layer): Áp dụng hàm kích hoạt (activation function) phi tuyến tính lên đầu ra của lớp tích chập để tạo ra các biểu diễn phi tuyến tính. Các hàm kích hoạt phổ biến bao gồm ReLU (Rectified Linear Unit), Sigmoid, Tanh.
- Lớp gộp (Pooling Layer): Giảm chiều dữ liệu bằng cách lấy giá trị lớn nhất (max pooling) hoặc giá trị trung bình (average pooling) trong một vùng lân cận.
- Lớp fully connected (Fully Connected Layer): Kết nối tất cả các nơ-ron ở lớp trước với tất cả các nơ-ron ở lớp sau, thường được sử dụng ở cuối mạng CNN để thực hiện phân loại.

2.1.2.2. Các kiến trúc mô hình liên quan

- ResNet50 (Residual Network 50):
 - Giới thiệu: ResNet50 là một kiến trúc CNN được giới thiệu bởi Kaiming He và cộng sự vào năm 2015. Điểm nổi bật của ResNet là sử dụng các "residual connections" (kết nối tắt) để giải quyết vấn đề "vanishing gradient" (độ dốc biến mất) trong các mạng nơ-ron sâu.

- Kiến trúc: ResNet50 có 50 lớp, bao gồm các lớp tích chập, lớp pooling và lớp fully connected. Các residual connections cho phép thông tin được truyền trực tiếp từ các lớp trước đến các lớp sau, giúp việc huấn luyện các mạng sâu dễ dàng hơn.
- Ưu điểm: ResNet50 có độ chính xác cao và đã đạt được kết quả tốt trên nhiều bộ dữ liệu ảnh khác nhau, bao gồm cả ImageNet.
- Ứng dụng trong đề tài: ResNet50 được sử dụng làm mô hình phân loại bệnh trên lá sầu, tận dụng khả năng trích xuất đặc trưng mạnh mẽ của nó.
- EfficientNetV2:
 - Giới thiệu: EfficientNetV2 là phiên bản cải tiến của EfficientNet, được giới thiệu bởi Mingxing Tan và Quoc V. Le vào năm 2021. EfficientNetV2 sử dụng kỹ thuật "compound scaling" (mở rộng hợp chất) để tối ưu hóa đồng thời chiều sâu (depth), chiều rộng (width) và độ phân giải (resolution) của mạng, đạt được hiệu suất cao hơn với số lượng tham số ít hơn.
 - Kiến trúc: EfficientNetV2 sử dụng kiến trúc MBConv (Mobile Inverted Bottleneck Convolution) làm khối xây dựng cơ bản. Nó cũng sử dụng các kỹ thuật tối ưu hóa như "progressive learning" (học tăng dần) và "stochastic depth" (độ sâu ngẫu nhiên).
 - Ưu điểm: EfficientNetV2 có độ chính xác cao hơn và tốc độ suy luận nhanh hơn so với EfficientNet.
 - Ứng dụng trong đề tài: EfficientNetV2 được sử dụng để phân loại bệnh trên lá sầu, với kỳ vọng đạt được độ chính xác cao và tốc độ suy luận nhanh, phù hợp cho triển khai thực tế.
- ViT (Vision Transformer):
 - Giới thiệu: ViT là một kiến trúc dựa trên mô hình Transformer, được giới thiệu bởi Alexey Dosovitskiy và cộng sự vào năm 2020. Khác với CNNs, ViT xử lý ảnh bằng cách chia ảnh thành các "patches" (miếng) và coi mỗi patch như một "token" (từ) trong mô hình Transformer.
 - Kiến trúc: ViT sử dụng các lớp "self-attention" (tự chú ý) để học mối quan hệ giữa các patches khác nhau trong ảnh.

- Ưu điểm: ViT có khả năng học các biểu diễn toàn cục của ảnh, không bị giới hạn bởi trường tiếp nhận (receptive field) như CNNs.
- Ứng dụng trong đề tài: ViT được sử dụng để phân loại bệnh trên lá sắn, tận dụng khả năng học biểu diễn toàn cục để phân biệt các loại bệnh có đặc trưng phức tạp.
- MobileNetV3:
 - Giới thiệu: MobileNetV3 là phiên bản thứ ba của MobileNet, được giới thiệu bởi Andrew Howard và cộng sự vào năm 2019. MobileNet được thiết kế để chạy trên các thiết bị di động và nhúng với tài nguyên tính toán hạn chế.
 - Kiến trúc: MobileNetV3 sử dụng các lớp "depthwise separable convolutions" (tích chập tách biệt theo chiều sâu) để giảm số lượng tham số và phép tính. Nó cũng sử dụng các khối "inverted residual" (đảo ngược phần dư) và "squeeze-and-excitation" (nén và kích thích).
 - Ưu điểm: MobileNetV3 có kích thước nhỏ gọn, tốc độ suy luận nhanh và độ chính xác tương đối tốt.
 - Ứng dụng trong đề tài: MobileNetV3 được sử dụng để phân loại bệnh trên lá sắn, hướng tới mục tiêu triển khai mô hình trên các thiết bị di động để người nông dân có thể sử dụng trực tiếp tại vườn.

2.1.2.3. YOLOv8 (You Only Look Once version 8)

- Giới thiệu: YOLOv8 là phiên bản mới nhất của mô hình YOLO (You Only Look Once), một mô hình phát hiện đối tượng thời gian thực (real-time object detection) nổi tiếng. YOLOv8 được phát triển bởi Ultralytics và được giới thiệu vào năm 2023.
- Kiến trúc: YOLOv8 sử dụng kiến trúc "anchor-free" (không sử dụng neo) và "decoupled head" (đầu tách rời) để cải thiện độ chính xác và tốc độ suy luận. Nó cũng sử dụng các kỹ thuật tối ưu hóa như "mosaic augmentation" (tăng cường dữ liệu bằng cách ghép ảnh) và "self-adversarial training" (huấn luyện tự đối kháng).
- Ưu điểm: YOLOv8 có tốc độ suy luận rất nhanh, độ chính xác cao và có thể phát hiện nhiều đối tượng trong ảnh cùng một lúc.

- Ứng dụng trong đề tài: Trong đề tài này, YOLOv8 được sử dụng để phát hiện lá sắn trong ảnh đầu vào. Sau khi xác định được vị trí của lá sắn, các vùng ảnh chứa lá sẽ được cắt ra và đưa vào các mô hình phân loại (ResNet50, EfficientNetV2, ViT, MobileNetV3) để xác định loại bệnh (nếu có).

2.1.3. Các kỹ thuật tăng cường dữ liệu (Data Augmentation)

Tăng cường dữ liệu là kỹ thuật tạo ra các mẫu dữ liệu mới từ dữ liệu gốc bằng cách áp dụng các phép biến đổi lên dữ liệu gốc. Mục đích của tăng cường dữ liệu là làm tăng kích thước và độ đa dạng của tập dữ liệu huấn luyện, giúp mô hình học được các đặc trưng tổng quát hơn và tránh hiện tượng overfitting.

2.1.3.1. Các kỹ thuật tăng cường dữ liệu cơ bản

- Xoay ảnh (Rotation): Xoay ảnh theo một góc ngẫu nhiên.
- Lật ảnh (Flipping): Lật ảnh theo chiều ngang hoặc chiều dọc.
- Cắt ảnh (Cropping): Cắt một phần ngẫu nhiên từ ảnh.
- Thay đổi kích thước (Resizing): Thay đổi kích thước ảnh.
- Thay đổi độ sáng (Brightness Adjustment): Tăng hoặc giảm độ sáng của ảnh.
- Thay đổi độ tương phản (Contrast Adjustment): Tăng hoặc giảm độ tương phản của ảnh.
- Thêm nhiễu (Noise Addition): Thêm nhiễu Gaussian hoặc các loại nhiễu khác vào ảnh.

2.1.3.2. Các kỹ thuật tăng cường dữ liệu nâng cao

Trong những năm gần đây, các kỹ thuật tăng cường dữ liệu nâng cao đã nổi lên như một phương pháp hiệu quả để cải thiện hiệu suất của các mô hình học sâu, đặc biệt là trong các trường hợp dữ liệu hạn chế. Các kỹ thuật này tạo ra các mẫu huấn luyện mới bằng cách kết hợp hoặc biến đổi các mẫu hiện có theo những cách phức tạp hơn so với các phương pháp tăng cường dữ liệu truyền thống. Dưới đây, chúng ta sẽ thảo luận về hai kỹ thuật phổ biến và hiệu quả: MixUp và CutMix.

a. MixUp

Giới

thiệu:

MixUp là một kỹ thuật tăng cường dữ liệu dựa trên nguyên tắc vicinal risk minimization (VRM), đề xuất bởi Hongyi Zhang và các cộng sự vào năm 2017.

Thay vì sử dụng các mẫu dữ liệu rời rạc, MixUp tạo ra các mẫu dữ liệu và nhãn tương ứng bằng cách kết hợp tuyến tính giữa hai mẫu ngẫu nhiên và nhãn của chúng. Điều này khuyến khích mô hình học cách ứng xử tuyến tính giữa các mẫu huấn luyện, dẫn đến việc cải thiện khả năng tổng quát hóa và hiệu chuẩn của mô hình.

Cách

thực

hiện:

Giả sử chúng ta có hai cặp dữ liệu ảnh và nhãn ngẫu nhiên được chọn từ tập huấn luyện: (x_i, y_i) và (x_j, y_j) , trong đó $x_i, x_j \in \mathbb{R}^{(W \times H \times C)}$ là các véc-tơ biểu diễn ảnh (với W, H, C lần lượt là chiều rộng, chiều cao và số kênh màu) và y_i, y_j là các véc-tơ one-hot biểu diễn nhãn tương ứng.

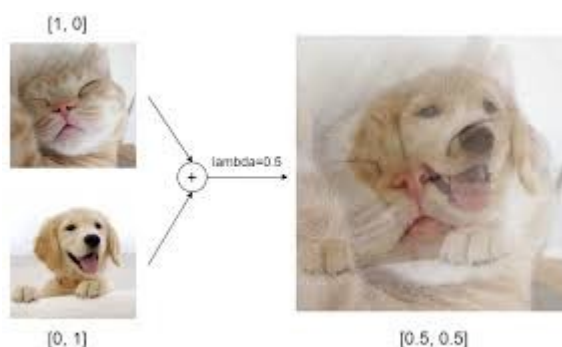
MixUp tạo ra một mẫu dữ liệu mới (x', y') như sau:

1. $x' = \lambda * x_i + (1 - \lambda) * x_j$
2. $y' = \lambda * y_i + (1 - \lambda) * y_j$

Trong đó, $\lambda \in [0, 1]$ là một hệ số trộn được lấy mẫu từ phân phối Beta với tham số α :

3. $\lambda \sim \text{Beta}(\alpha, \alpha)$

Tham số α là một siêu tham số kiểm soát mức độ trộn giữa hai mẫu. Khi $\alpha \rightarrow 0$, MixUp gần giống với việc không tăng cường dữ liệu. Khi $\alpha \rightarrow \infty$, λ có xu hướng tập trung quanh 0.5, dẫn đến sự kết hợp mạnh mẽ giữa hai mẫu. Giá trị α thường được chọn trong khoảng $[0.1, 0.4]$.



Hình 2.1. Ảnh minh họa Mixup

Ưu điểm:

- **Tính đơn giản và hiệu quả:** MixUp rất dễ cài đặt và có thể áp dụng cho nhiều loại dữ liệu khác nhau.
 - **Cải thiện khả năng tổng quát hóa:** Bằng cách khuyến khích mô hình học cách ứng xử tuyến tính giữa các mẫu huấn luyện, MixUp giúp mô hình tổng quát hóa tốt hơn trên dữ liệu chưa từng thấy.
 - **Tăng cường độ chắc chắn của mô hình:** MixUp đã được chứng minh là giúp mô hình chống chịu tốt hơn với nhiễu và các cuộc tấn công đối nghịch (adversarial attacks).
 - **Hiệu chuẩn tốt hơn:** MixUp giúp giảm thiểu hiện tượng overconfidence (quá tự tin) của mô hình, dẫn đến việc ước lượng xác suất tin cậy hơn.
-

b. CutMix

Giới

thiệu:

CutMix, được đề xuất bởi Sangdoo Yun và các cộng sự vào năm 2019, là một kỹ thuật tăng cường dữ liệu kết hợp ý tưởng của MixUp và Cutout. Thay vì trộn toàn bộ hai ảnh, CutMix cắt một vùng hình chữ nhật từ một ảnh và dán nó vào vị trí ngẫu nhiên trên ảnh khác. Nhãn của mẫu mới được tạo ra bằng cách kết hợp tuyến tính nhãn của hai ảnh ban đầu, theo tỷ lệ diện tích của vùng cắt. Kỹ thuật này khuyến khích mô hình học cách nhận diện đối tượng dựa trên các phần cục bộ, thay vì dựa vào toàn bộ ngữ cảnh của ảnh.

Cách

thực

hiện:

Tương tự như MixUp, giả sử chúng ta có hai cặp dữ liệu ảnh và nhãn ngẫu nhiên: (x_i, y_i) và (x_j, y_j) .

CutMix tạo ra một mẫu dữ liệu mới (x', y') như sau:

1. $x' = M \odot x_i + (1 - M) \odot x_j$
2. $y' = \lambda * y_i + (1 - \lambda) * y_j$

Trong đó:

- $\mathbf{M} \in \{0, 1\}^{(W \times H)}$ là một ma trận nhị phân, chỉ định vị trí của vùng cắt. Các phần tử $M_{r,c} = 1$ biểu thị các pixel được giữ lại từ ảnh x_i , và $M_{r,c} = 0$ biểu thị các pixel được thay thế bằng các pixel tương ứng từ ảnh x_j .
- $\odot$ là phép nhân Hadamard (nhân từng phần tử).
- $\mathbf{1}$ là ma trận có cùng kích thước với $\mathbf{M}$, nhưng tất cả các phần tử đều bằng 1.
- λ là tỷ lệ diện tích của vùng cắt so với toàn bộ ảnh, được tính như sau:

$$\lambda = 1 - (W_c * H_c) / (W * H)$$

Trong đó:

- W_c và H_c là chiều rộng và chiều cao của vùng cắt.
- W và H là chiều rộng và chiều cao của ảnh.

Để tạo ma trận $\mathbf{M}$, ta chọn ngẫu nhiên một hộp giới hạn $B = (r_x, r_y, r_w, r_h)$ sao cho:

- $r_x \sim \text{Uniform}(\{0, \dots, W\})$, $r_y \sim \text{Uniform}(\{0, \dots, H\})$
- $r_w = W * \sqrt{1-\lambda}$, $r_h = H * \sqrt{1-\lambda}$
- $r_x + r_w \leq W$, $r_y + r_h \leq H$

Sau đó, ta gán:

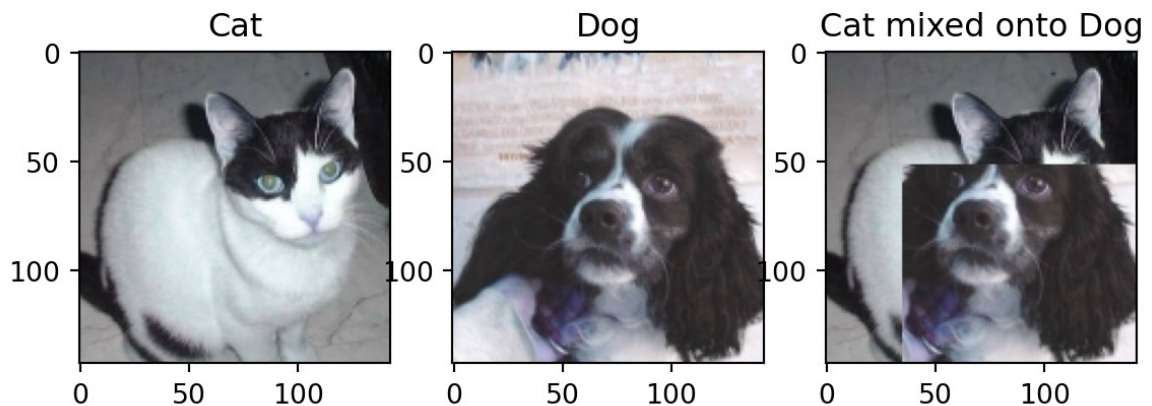
- $M_{r,c} = 0$ nếu $r_x \leq r \leq r_x + r_w$ và $r_y \leq c \leq r_y + r_h$
- $M_{r,c} = 1$ trong các trường hợp khác.

Ưu điểm:

- **Học các đặc trưng cục bộ:** CutMix buộc mô hình phải chú ý đến các phần nhỏ hơn của đối tượng để phân loại chính xác, thay vì chỉ dựa vào các đặc trưng toàn cục.
- **Cải thiện khả năng định vị đối tượng:** Do phải học cách nhận diện đối tượng từ các phần bị che khuất, CutMix gián tiếp cải thiện khả năng định vị đối tượng của mô hình.

- **Chống chịu với nhiễu và che khuất:** CutMix giúp mô hình trở nên chắc chắn hơn đối với các trường hợp ảnh bị nhiễu hoặc bị che khuất một phần.

Cutmix example



Hình 2.2. Ảnh minh họa cutmix

2.1.4. Các phương pháp nâng cao hiệu quả của mô hình

2.1.4.1. Độ chính xác (Accuracy)

- **Tăng cường dữ liệu (Data Augmentation):** Như đã đề cập ở phần 2.1.3, tăng cường dữ liệu giúp tăng kích thước và độ đa dạng của tập dữ liệu huấn luyện, dẫn đến mô hình học được các đặc trưng tổng quát hơn và cải thiện độ chính xác.
- **Tinh chỉnh siêu tham số (Hyperparameter Tuning):** Các siêu tham số như learning rate, batch size, số lượng epoch, optimizer ảnh hưởng trực tiếp đến hiệu quả huấn luyện của mô hình. Việc tinh chỉnh các siêu tham số này để tìm ra bộ siêu tham số tối ưu có thể giúp cải thiện đáng kể độ chính xác của mô hình. Các phương pháp tinh chỉnh siêu tham số phổ biến bao gồm Grid Search, Random Search, Bayesian Optimization.
- **Transfer Learning:** Sử dụng các mô hình đã được huấn luyện trước (pretrained models) trên các tập dữ liệu lớn (như ImageNet) và tinh chỉnh (fine-tune) chúng trên tập dữ liệu của mình. Việc này giúp tận dụng các đặc trưng đã học được từ tập dữ liệu lớn, giúp mô hình đạt độ chính xác cao hơn, đặc biệt khi tập dữ liệu của bạn nhỏ.

- Regularization: Sử dụng các kỹ thuật như L1/L2 regularization, Dropout để tránh overfitting, giúp mô hình tổng quát hóa tốt hơn trên tập dữ liệu kiểm tra.
- Ensemble Learning: Kết hợp nhiều mô hình khác nhau để đưa ra dự đoán cuối cùng. Việc này có thể giúp cải thiện độ chính xác và độ ổn định của mô hình.

2.1.4.2. Thời gian suy luận (Inference Time)

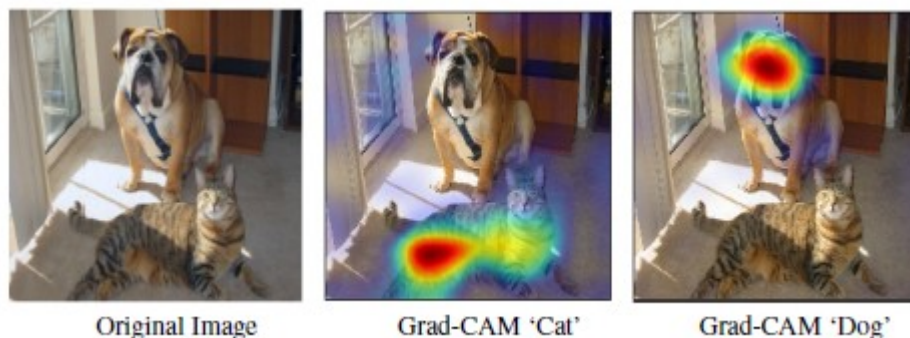
- Lựa chọn kiến trúc mô hình: Một số kiến trúc mô hình được thiết kế để có tốc độ suy luận nhanh, chẳng hạn như MobileNet. Việc lựa chọn kiến trúc mô hình phù hợp với yêu cầu về tốc độ suy luận là rất quan trọng.
- Tối ưu hóa mô hình (Model Optimization):
 - Pruning: Loại bỏ các kết nối (weights) không quan trọng trong mô hình để giảm kích thước mô hình và tăng tốc độ suy luận.
 - Quantization: Giảm độ chính xác của các trọng số (weights) trong mô hình (ví dụ: từ float32 sang int8) để giảm kích thước mô hình và tăng tốc độ suy luận.
 - Knowledge Distillation: Huấn luyện một mô hình nhỏ hơn (student model) để bắt chước đầu ra của một mô hình lớn hơn (teacher model). Mô hình nhỏ hơn này sẽ có tốc độ suy luận nhanh hơn mà vẫn giữ được độ chính xác tương đối tốt.
- Tối ưu hóa phần cứng (Hardware Optimization): Sử dụng GPU hoặc các phần cứng chuyên dụng khác để tăng tốc độ suy luận.
- Batching: Gộp nhiều ảnh đầu vào thành một batch và đưa vào mô hình cùng một lúc để tận dụng khả năng tính toán song song của GPU, giúp tăng tốc độ suy luận.

2.1.5. Explainable AI (XAI) - Trực quan hóa kết quả AI

Explainable AI (XAI) là một lĩnh vực nghiên cứu mới nổi trong AI, tập trung vào việc làm cho các mô hình AI trở nên minh bạch và dễ hiểu hơn. Trong lĩnh vực thị giác máy tính, XAI thường được sử dụng để trực quan hóa các vùng ảnh mà mô hình tập trung vào khi đưa ra dự đoán, giúp con người hiểu được cách thức hoạt động của mô hình và lý giải các kết quả dự đoán.

2.1.5.1. Grad-CAM (Gradient-weighted Class Activation Mapping)

- Giới thiệu: Grad-CAM là một kỹ thuật XAI phổ biến, sử dụng gradients của đầu ra dự đoán đối với các feature maps của lớp tích chập cuối cùng để tạo ra một heatmap (bản đồ nhiệt) cho biết mức độ quan trọng của các vùng khác nhau trong ảnh đối với dự đoán của mô hình.
- Cách thực hiện:
 - Tính gradients của đầu ra dự đoán (ví dụ: xác suất của lớp "bệnh khảm lá") đối với các feature maps của lớp tích chập cuối cùng.
 - Tính trung bình có trọng số của các feature maps theo gradients tương ứng.
 - Áp dụng hàm ReLU lên kết quả trung bình có trọng số để loại bỏ các giá trị âm.
 - Upsample heatmap về kích thước của ảnh gốc và chồng lên ảnh gốc để trực quan hóa.
- Ứng dụng trong đề tài: Grad-CAM được sử dụng để trực quan hóa các vùng lá sắn mà mô hình tập trung vào khi phân loại bệnh. Việc này giúp:
 - Kiểm tra xem mô hình có đang học đúng các đặc trưng của bệnh hay không: Ví dụ, nếu mô hình phân loại sai một ảnh, Grad-CAM có thể cho biết liệu mô hình có đang tập trung vào các vùng không liên quan hay không.
 - Giải thích các trường hợp đúng/sai: Grad-CAM giúp hiểu rõ hơn tại sao mô hình lại đưa ra dự đoán đúng hoặc sai trong từng trường hợp cụ thể.
 - Tăng độ tin cậy của mô hình: Việc minh họa cách thức hoạt động của mô hình giúp tăng độ tin cậy và sự tin tưởng của người dùng vào kết quả dự đoán.



Hình 2.3. Ảnh minh họa grad cam

2.1.5.2. Các phương pháp XAI khác

Ngoài Grad-CAM, còn có nhiều phương pháp XAI khác có thể được sử dụng để giải thích các mô hình phân loại ảnh, bao gồm:

- LIME (Local Interpretable Model-agnostic Explanations): Tạo ra các giải thích cục bộ cho từng dự đoán bằng cách xấp xỉ mô hình phức tạp bằng một mô hình đơn giản hơn (ví dụ: mô hình tuyến tính) trong vùng lân cận của điểm dữ liệu cần giải thích.
- SHAP (SHapley Additive exPlanations): Sử dụng lý thuyết trò chơi để tính toán mức độ đóng góp của từng đặc trưng (pixel) vào dự đoán của mô hình.
- SmoothGrad: Giảm nhiễu trong các bản đồ kích hoạt (activation maps) bằng cách lấy trung bình các gradients từ nhiều phiên bản nhiễu của ảnh đầu vào.

2.2. Các nghiên cứu liên quan

2.2.1. Tổng quan về ứng dụng học sâu trong nông nghiệp

Trong những năm gần đây, học sâu đã được ứng dụng rộng rãi trong nông nghiệp và đạt được nhiều kết quả ấn tượng. Một số hướng ứng dụng chính bao gồm:

- Nhận dạng cây trồng (Crop Recognition): Sử dụng các mô hình học sâu để phân loại các loại cây trồng khác nhau dựa trên ảnh chụp từ vệ tinh, máy bay không người lái hoặc camera.
- Phát hiện sâu bệnh hại (Pest and Disease Detection): Sử dụng các mô hình học sâu để phát hiện và phân loại các loại sâu bệnh hại trên cây trồng dựa trên ảnh chụp lá, thân, quả,...
- Dự đoán năng suất cây trồng (Yield Prediction): Sử dụng các mô hình học sâu để dự đoán năng suất cây trồng dựa trên các yếu tố như thời tiết, chất lượng đất, hình ảnh cây trồng,...
- Phân loại chất lượng nông sản (Crop Quality Classification): Sử dụng các mô hình học sâu để phân loại chất lượng nông sản dựa trên hình ảnh, kích thước, màu sắc,...
- Giám sát tưới tiêu (Irrigation Monitoring): Sử dụng các mô hình học sâu để phân tích dữ liệu từ các cảm biến độ ẩm đất, hình ảnh cây trồng để tối ưu hóa việc tưới tiêu.

2.2.2. Các nghiên cứu về nhận dạng sâu bệnh hại cây trồng

Đã có nhiều nghiên cứu về việc ứng dụng học sâu để nhận dạng sâu bệnh hại cây trồng, sử dụng các loại cây trồng và các loại bệnh khác nhau:

- Ferentinos (2018) đã sử dụng các mô hình CNN (AlexNet, GoogLeNet, VGG) để phân loại 58 loại bệnh trên 25 loại cây trồng khác nhau, đạt độ chính xác 99.53% trên tập dữ liệu PlantVillage.
- Mohanty et al. (2016) đã sử dụng GoogLeNet để phân loại 26 loại bệnh trên 14 loại cây trồng, đạt độ chính xác 99.35% trên tập dữ liệu PlantVillage.
- Too et al. (2018) đã so sánh hiệu quả của các mô hình CNN (VGG16, InceptionV3, ResNet50, DenseNet121) trong việc phân loại bệnh trên lá cà chua, đạt độ chính xác cao nhất với DenseNet121 (99.75%).
- Barbedo (2018) đã phân tích các yếu tố ảnh hưởng đến hiệu quả của các mô hình CNN trong việc nhận dạng bệnh trên lá cây trồng, bao gồm: số lượng mẫu huấn luyện, loại bệnh, phương pháp tăng cường dữ liệu.
- Atila et al. (2021) đã sử dụng EfficientNet để phân loại bệnh trên lá cà chua và ớt chuông và phát triển một ứng dụng di động để nông dân sử dụng, đạt độ chính xác 99.67%.

2.2.3. Các nghiên cứu về nhận dạng bệnh trên cây sắn

Một số nghiên cứu đã tập trung vào việc nhận dạng bệnh trên cây sắn sử dụng học sâu:

2.2.4. Đánh giá các nghiên cứu và điểm cải tiến trong đề tài

Các nghiên cứu trước đây đã cho thấy tiềm năng to lớn của việc ứng dụng học sâu trong việc nhận dạng sâu bệnh hại cây trồng, bao gồm cả cây sắn. Tuy nhiên, hầu hết các nghiên cứu này mới chỉ tập trung vào việc phân loại bệnh dựa trên ảnh chụp lá đã được cắt xén sẵn, chưa tích hợp với mô-đun phát hiện đối tượng.

Điểm cải tiến trong đề tài này là:

- Nghiên cứu trên tập dữ liệu riêng: Bên cạnh việc sử dụng tập dữ liệu từ Kaggle, đề tài còn thu thập và xây dựng thêm bộ dữ liệu ảnh về các loại sâu bệnh hại chính trên cây sắn tại Việt Nam. Việc này giúp mô hình học được các đặc trưng của sâu bệnh hại trong điều kiện thực tế tại Việt Nam, nâng cao độ chính xác và khả năng ứng dụng của hệ thống.

- Có sử dụng yolo để xác định ảnh user chụp có cây khoai mì hay không ? và ước lượng khoảng cách chụp hợp lý cho user.
- Tập trung vào các loại bệnh phổ biến tại Việt Nam: Đề tài tập trung vào 4 loại bệnh phổ biến và gây hại nghiêm trọng đến cây sắn ở Việt Nam: đốm lá, cháy lá, khảm lá do virus, thối rễ và tình trạng khỏe mạnh.
- So sánh và đánh giá nhiều mô hình: Đề tài tiến hành so sánh và đánh giá hiệu quả của nhiều mô hình học sâu khác nhau (ResNet50, EfficientNetV2, ViT, MobileNetV3) để tìm ra mô hình phù hợp nhất cho bài toán nhận dạng sâu bệnh hại cây sắn.
- Áp dụng các kỹ thuật tăng cường dữ liệu nâng cao: Ngoài các kỹ thuật tăng cường dữ liệu cơ bản, đề tài còn áp dụng các kỹ thuật nâng cao như MixUp và CutMix để nâng cao hiệu quả của mô hình.
- Tối ưu hóa mô hình cho triển khai thực tế: Đề tài chú trọng đến việc tối ưu hóa mô hình để giảm thời gian suy luận, hướng tới triển khai trên các thiết bị di động, giúp người nông dân có thể sử dụng trực tiếp tại vườn.
- Sử dụng XAI để giải thích kết quả: Đề tài sử dụng Grad-CAM để trực quan hóa các vùng ảnh mà mô hình tập trung vào khi phân loại bệnh, giúp tăng độ tin cậy và khả năng giải thích của mô hình.

2.3. Công cụ và môi trường phát triển

2.3.1. Ngôn ngữ lập trình Python

Python là ngôn ngữ lập trình hướng đối tượng, bậc cao, được sử dụng rộng rãi trong lĩnh vực khoa học dữ liệu và học máy. Python có cú pháp đơn giản, dễ học, dễ đọc và có một hệ sinh thái các thư viện phong phú, hỗ trợ mạnh mẽ cho việc phát triển các ứng dụng học sâu. Trong nghiên cứu này, Python được sử dụng làm ngôn ngữ lập trình chính để triển khai các thuật toán, xây dựng và huấn luyện mô hình, cũng như xử lý và phân tích dữ liệu.

2.3.2. Thư viện PyTorch và Timm

- PyTorch: Là một thư viện mã nguồn mở được phát triển bởi Facebook, cung cấp các công cụ mạnh mẽ và linh hoạt để xây dựng và huấn luyện các mô hình học sâu. PyTorch sử dụng đồ thị tính toán động (dynamic computation graphs), giúp cho việc phát triển và thử nghiệm các mô hình mới trở nên dễ dàng và trực quan hơn. PyTorch cũng hỗ trợ tính toán trên CPU và GPU, cho

phép tận dụng tối đa sức mạnh phần cứng để tăng tốc quá trình huấn luyện. Trong nghiên cứu này, PyTorch được sử dụng làm framework chính để xây dựng, huấn luyện và đánh giá các mô hình học sâu.

- Timm (PyTorch Image Models): Là một thư viện mã nguồn mở cung cấp các bộ sưu tập lớn các mô hình học sâu đã được huấn luyện trước (pretrained models) trên tập dữ liệu ImageNet, cũng như các công cụ hữu ích để huấn luyện và đánh giá các mô hình thị giác máy tính trong PyTorch. Timm cung cấp các kiến trúc mô hình tiên tiến như ResNet, EfficientNet, ViT, MobileNet, v.v. Trong nghiên cứu này, Timm được sử dụng để tải và sử dụng các mô hình đã được huấn luyện trước, cũng như tận dụng các tiện ích của nó để tinh chỉnh (fine-tune) và tối ưu hóa mô hình cho bài toán nhận dạng sâu bệnh hại cây sắn.

2.3.3. Thư viện OpenCV

OpenCV (Open Source Computer Vision Library) là một thư viện mã nguồn mở chuyên về thị giác máy tính và xử lý ảnh. OpenCV cung cấp các hàm để thực hiện các thao tác xử lý ảnh cơ bản như đọc, ghi ảnh, chuyển đổi không gian màu, lọc nhiễu, phát hiện biên, thay đổi kích thước ảnh, v.v. Trong nghiên cứu này, OpenCV được sử dụng chủ yếu trong giai đoạn tiền xử lý dữ liệu, giúp chuẩn hóa dữ liệu đầu vào và tăng cường chất lượng ảnh.

2.3.4. Thư viện Scikit-learn

Scikit-learn là một thư viện mã nguồn mở cung cấp các công cụ cho học máy, bao gồm các thuật toán phân loại, hồi quy, gom cụm, giảm chiều dữ liệu, lựa chọn mô hình và tiền xử lý dữ liệu. Trong nghiên cứu này, Scikit-learn được sử dụng để đánh giá hiệu năng mô hình, bao gồm các độ đo như accuracy, precision, recall, F1-score, confusion matrix.

2.3.5. Nền tảng Google Colab

Google Colab (Colaboratory) là một dịch vụ miễn phí của Google cung cấp môi trường lập trình Jupyter Notebook trên nền tảng đám mây, cho phép người dùng viết và thực thi mã Python trực tiếp trên trình duyệt. Colab cung cấp tài nguyên tính toán miễn phí, bao gồm CPU, GPU và TPU, giúp tăng tốc quá trình huấn luyện các mô hình học sâu. Trong nghiên cứu này, Google Colab được sử dụng để tận dụng

sức mạnh của GPU, giúp rút ngắn thời gian huấn luyện các mô hình, đặc biệt là các mô hình lớn và phức tạp.

2.3.6. Thư viện Matplotlib và Seaborn

- Matplotlib: Là một thư viện Python dùng để vẽ đồ thị và biểu đồ. Matplotlib cung cấp các hàm để tạo ra các biểu đồ đường, biểu đồ cột, biểu đồ phân tán, biểu đồ histogram, v.v.
- Seaborn: Là một thư viện Python được xây dựng dựa trên Matplotlib, cung cấp giao diện bậc cao để vẽ các biểu đồ thống kê đẹp và mang nhiều thông tin. Seaborn giúp đơn giản hóa việc tạo ra các biểu đồ phức tạp, đồng thời cung cấp các kiểu biểu đồ và bảng màu hấp dẫn.

Trong nghiên cứu này, Matplotlib và Seaborn được sử dụng để trực quan hóa dữ liệu, biểu diễn kết quả huấn luyện và đánh giá mô hình, giúp cho việc phân tích và diễn giải kết quả trở nên dễ dàng và trực quan hơn.

2.3.7. Công cụ gán nhãn dữ liệu: MakeSense.ai

MakeSense.ai là một công cụ gán nhãn dữ liệu trực tuyến và miễn phí, được thiết kế để hỗ trợ các bài toán phát hiện đối tượng một cách hiệu quả và trực quan. Công cụ này cung cấp giao diện thân thiện với người dùng, hỗ trợ các định dạng nhãn phổ biến như Pascal VOC và COCO, rất phù hợp với các mô hình như YOLO và các phương pháp học sâu khác. Trong nghiên cứu này, MakeSense.ai được sử dụng để gán nhãn dữ liệu ảnh tự chụp về lá sắn. Các bounding boxes và nhãn được tạo thủ công cho từng đối tượng lá sắn trong ảnh, đảm bảo chất lượng dữ liệu đầu vào cho quá trình huấn luyện mô hình YOLOv8. Công cụ này giúp giảm thiểu thời gian chuẩn bị dữ liệu và đảm bảo tính chính xác trong việc đánh dấu các đối tượng.

2.3.8. Môi trường phần cứng và nền tảng huấn luyện

- GPU (Graphics Processing Unit): Quá trình huấn luyện các mô hình học sâu, đặc biệt là các mô hình phức tạp như YOLOv8, yêu cầu năng lực tính toán cao. GPU đóng vai trò quan trọng trong việc tăng tốc độ huấn luyện nhờ khả năng xử lý song song hiệu quả. Nghiên cứu này sử dụng Google Colab, một nền tảng trực tuyến phổ biến, cung cấp GPU miễn phí. GPU trên Colab giúp giảm thời gian huấn luyện mô

hình đáng kể, đảm bảo tính khả thi cho các thí nghiệm ngay cả với các tài nguyên hạn chế.

- CPU (Central Processing Unit):
Mặc dù GPU chịu trách nhiệm chính trong việc tăng tốc các tác vụ học sâu, CPU vẫn đóng vai trò thiết yếu trong các công đoạn xử lý dữ liệu, quản lý luồng công việc, và thực hiện các tác vụ không yêu cầu tính toán song song. Sự kết hợp giữa CPU và GPU trong Google Colab giúp tối ưu hóa hiệu suất tổng thể của hệ thống.
- Bộ nhớ (RAM):
Dung lượng RAM cũng là một yếu tố quan trọng ảnh hưởng đến khả năng huấn luyện các mô hình học sâu. Việc xử lý các tập dữ liệu lớn và mô hình phức tạp đòi hỏi dung lượng RAM đủ lớn để lưu trữ tạm thời dữ liệu trong quá trình huấn luyện. Google Colab cung cấp một lượng RAM tương đối lớn, đủ để thực hiện các thí nghiệm trong nghiên cứu này. Điều này đảm bảo rằng hệ thống không bị nghẽn trong quá trình xử lý, từ đó tăng cường hiệu quả huấn luyện.

Nhờ sử dụng MakeSense.ai và Google Colab, nghiên cứu đạt được sự cân bằng giữa hiệu quả, chi phí và tính linh hoạt, giúp quá trình phát triển mô hình trở nên dễ dàng và hiệu quả hơn.

CHƯƠNG 3: PHƯƠNG PHÁP VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày chi tiết về phương pháp nghiên cứu, quy trình thu thập và xử lý dữ liệu, các mô hình học sâu được sử dụng, cũng như thiết kế tổng thể của hệ thống nhận dạng sâu bệnh hại cây sắn.

3.1. Phương pháp nghiên cứu

Nghiên cứu này sử dụng phương pháp nghiên cứu định lượng, kết hợp với các kỹ thuật học sâu để xây dựng mô hình nhận dạng sâu bệnh hại cây sắn. Quy trình nghiên cứu bao gồm các bước chính sau:

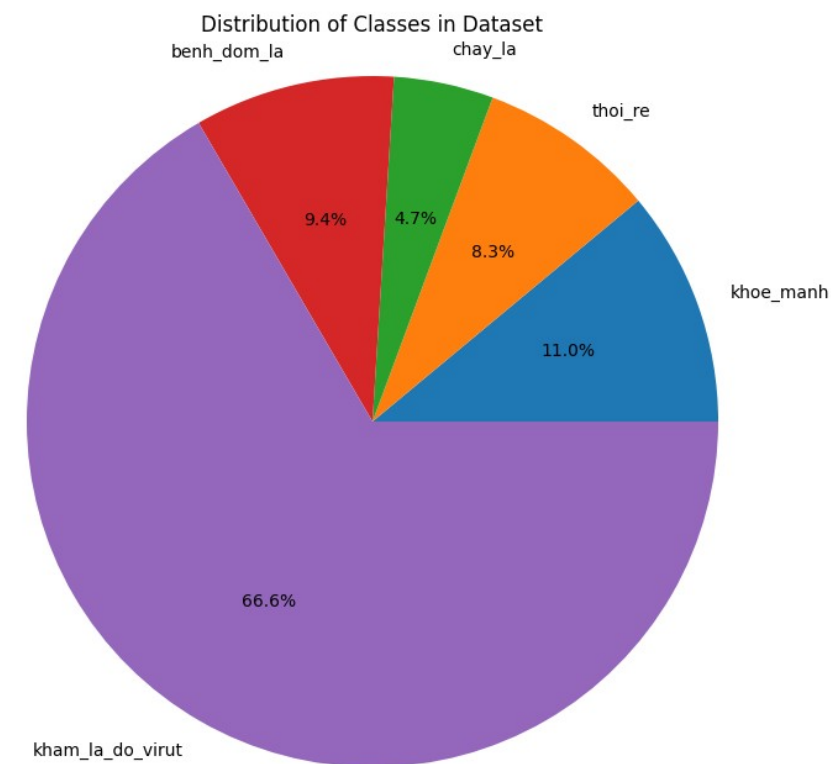
1. Thu thập dữ liệu: Thu thập dữ liệu ảnh về các loại sâu bệnh hại cây sắn từ nguồn Kaggle và ảnh tự chụp tại một số vùng trồng sắn ở Việt Nam.
2. Gán nhãn dữ liệu: Sử dụng công cụ Makesense.ai để gán nhãn dữ liệu cho bài toán phát hiện đối tượng (object detection) và phân loại ảnh (image classification).
3. Tiền xử lý dữ liệu: Áp dụng các kỹ thuật tiền xử lý ảnh để chuẩn hóa dữ liệu đầu vào, bao gồm: thay đổi kích thước ảnh, chuẩn hóa, tăng cường dữ liệu.
4. Huấn luyện mô hình:
 - Huấn luyện mô hình YOLOv8 để phát hiện lá sắn trong ảnh.
 - Huấn luyện các mô hình phân loại (ResNet50, EfficientNetV2, ViT, MobileNetV3) để nhận dạng các loại sâu bệnh hại trên lá sắn.
5. Đánh giá mô hình: Đánh giá hiệu quả của các mô hình dựa trên các độ đo phù hợp: accuracy, precision, recall, F1-score, mAP.
6. Tối ưu hóa mô hình: Áp dụng các kỹ thuật tối ưu hóa để nâng cao hiệu quả của mô hình, bao gồm: tinh chỉnh siêu tham số, transfer learning.
7. Lý giải mô hình: Áp dụng grad-cam để minh họa về lý do mô hình đưa ra kết quả như vậy và giải thích các trường hợp đúng, sai của mô hình.
8. Thử nghiệm khả năng chống chịu của mô hình bằng các phương pháp trộn nhiễu vào các ảnh thử nghiệm.
9. Xây dựng hệ thống demo: Thiết kế và xây dựng hệ thống demo cho phép người dùng tải lên ảnh lá sắn và nhận được kết quả chẩn đoán.
10. Đánh giá khả năng triển khai thực tế: Thử nghiệm hệ thống trên các ảnh chụp thực tế và đánh giá tốc độ, độ chính xác trong điều kiện thực tế.

3.2. Thu thập và xử lý dữ liệu

3.2.1. Mô tả dataset

Bộ dữ liệu sử dụng trong nghiên cứu này bao gồm hai phần:

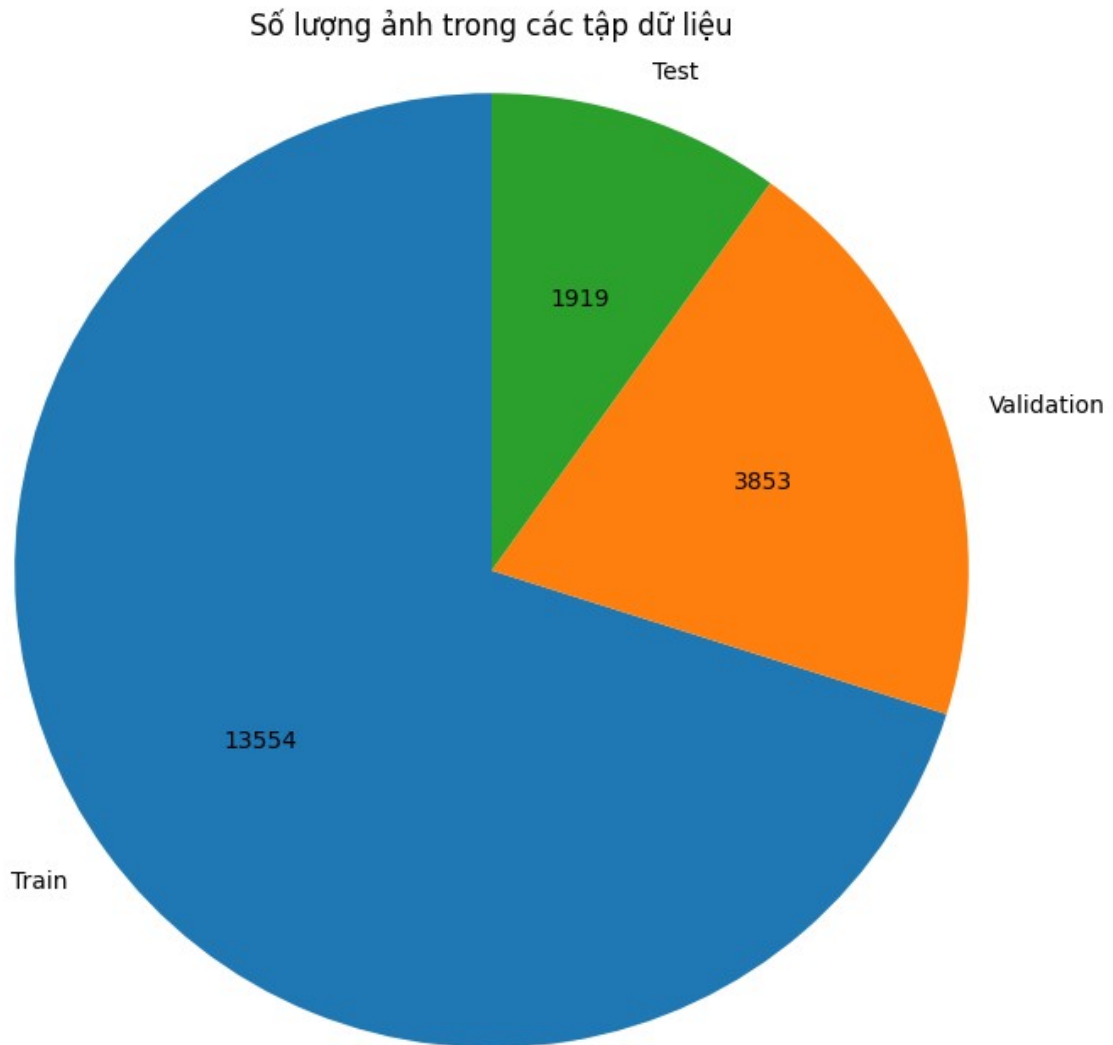
- Dataset cho bài toán phân loại (classification):
 - Nguồn gốc:
 - Kaggle: Sử dụng bộ dữ liệu "Cassava Leaf Disease Classification" trên Kaggle làm nguồn dữ liệu chính. Bộ dữ liệu này chứa ảnh của 4 loại bệnh.
 - Ảnh tự chụp: Thu thập thêm ảnh về bệnh thối rễ trên cây sắn tại một số vùng trồng sắn ở Việt Nam .
 - Số lượng ảnh là 23,335 ảnh phân bố cho 5 class như hình bên dưới



Hình 3.1. Bảng thống kê số lượng ảnh từng class

- Độ phân giải: Các ảnh có độ phân giải và kích thước khác nhau.
- Đặc điểm: Các ảnh được chụp trong các điều kiện ánh sáng, góc chụp và mức độ bệnh khác nhau, bao gồm cận cảnh lá bị bệnh và toàn bộ cây sắn.
- Dataset cho bài toán phát hiện đối tượng (object detection):
 - Nguồn gốc: Tự xây dựng bằng cách chụp ảnh các ruộng sắn và cây sắn ở các vùng [Liệt kê cụ thể các địa điểm].

- Số lượng: Tổng cộng 19326 ảnh, được chia thành 3 tập:
 - Train: 13554 ảnh
 - Validation: 3853 ảnh
 - Test: 1919 ảnh



Hình 3.2: Biểu đồ tròn thể hiện tỷ lệ ảnh trong các tập train, validation và test cho tập Object Detection.

- Độ phân giải: Các ảnh có độ phân giải và kích thước khác nhau.
- Đặc điểm: Các ảnh chứa các cây sắn trong nhiều điều kiện môi trường khác nhau, bao gồm nhiều lá sắn trong một ảnh. Mỗi lá sắn trong ảnh sẽ được gán nhãn "lá sắn" (cassava leaf).
- Class: Chỉ có 1 class duy nhất là "lá sắn".

3.2.2. Gán nhãn dữ liệu

Việc gán nhãn dữ liệu đóng vai trò then chốt trong việc xây dựng các mô hình học máy hiệu quả. Trong nghiên cứu này, chúng tôi tiến hành gán nhãn cho hai loại bài toán chính: phân loại bệnh sắn (Cassava Leaf Disease Classification) và phát hiện lá sắn (Cassava Leaf Detection).

3.2.2.1. Dữ liệu cho bài toán phân loại bệnh sắn (Cassava Leaf Disease Classification)

Tập dữ liệu cho bài toán phân loại bệnh sắn được thu thập từ hai nguồn chính:

- Tập dữ liệu từ Kaggle: Tập dữ liệu "Cassava Leaf Disease Classification" trên Kaggle cung cấp hình ảnh lá sắn đã được gán nhãn sẵn. Tuy nhiên, nhãn ban đầu sử dụng tiếng Anh, bao gồm "cbb", "cbsd", "cgm", "cmd", và "healthy". Để phù hợp với ngữ cảnh Việt Nam và tăng tính trực quan, chúng tôi đã tiến hành quy đổi các nhãn này sang tiếng Việt như sau:
 1. "cbb" (Cassava Bacterial Blight) được chuyển thành "chay_la".
 2. "cbsd" (Cassava Brown Streak Disease) được chuyển thành "benh_dom_la".
 3. "cgm" (Cassava Green Mottle) được chuyển thành "kham_la_do_virut".
 4. "cmd" (Cassava Mosaic Disease) được chuyển thành "kham_la_do_virut".
 5. "healthy" được chuyển thành "khoe_manh".
- Tập dữ liệu tự thu thập: Ngoài tập dữ liệu từ Kaggle, chúng tôi tiến hành thu thập thêm hình ảnh lá sắn tại các khu vực trồng sắn ở Việt Nam. Những hình ảnh này sau đó được phân loại thủ công dựa vào đặc điểm bệnh lý và gán nhãn bằng cách sắp xếp vào các thư mục con tương ứng trong thư mục "dataset" chính.

Quy trình gán nhãn và phân loại cho dữ liệu tự thu thập:

B1. Kiểm tra và phân loại: Dựa vào đặc điểm hình thái của lá sắn trên từng ảnh, các chuyên gia nông nghiệp hoặc người có kinh nghiệm sẽ tiến hành kiểm tra và xác định loại bệnh (hoặc lá khỏe mạnh) tương ứng.

B2. Tạo thư mục con: Trong thư mục "dataset" chính, các thư mục con được tạo ra để đại diện cho từng loại nhãn, bao gồm: chay_la, benh_dom_la, kham_la_do_virut, thoi_re, và khoe_manh.

B3. Gán nhãn bằng cách di chuyển ảnh: Mỗi ảnh sau khi được kiểm tra và xác định loại bệnh sẽ được di chuyển vào thư mục con tương ứng. Ví dụ, ảnh lá sắn bị bệnh thối rữa sẽ được đưa vào thư mục "thoi_re". Như vậy, tên của thư mục con chính là nhãn của các hình ảnh bên trong nó.

3.2.2.2. Dữ liệu cho bài toán phát hiện lá sắn (Cassava Leaf Detection)

Đối với bài toán phát hiện lá sắn, mục tiêu là xác định vị trí của tất cả các lá sắn trong ảnh, bất kể tình trạng sức khỏe của lá. Do đó, tập dữ liệu cho bài toán này chỉ sử dụng một nhãn duy nhất là "la_san".

Quy trình gán nhãn cho bài toán phát hiện lá sắn:

1. Tải ảnh lên: Tải các hình ảnh chứa lá sắn lên nền tảng Makesense.ai.
2. Chọn nhãn: Chọn nhãn "la_san".
3. Vẽ bounding box: Sử dụng công cụ vẽ khung bao (bounding box) của Makesense.ai để khoanh vùng chính xác vị trí của từng lá sắn riêng biệt trong ảnh. Mỗi bounding box sẽ bao quanh toàn bộ lá sắn và được gán nhãn "la_san".
4. Xuất nhãn: Xuất các nhãn đã được gán dưới định dạng YOLO format, tương thích với kiến trúc YOLOv8 được sử dụng trong nghiên cứu này. Định dạng này bao gồm thông tin về vị trí (tọa độ tâm, chiều rộng, chiều cao) và nhãn của bounding box.



Hình 3.4: Ảnh minh họa gán nhãn "la_san" cho dataset object detection.

3.2.3. Phân tích dữ liệu (Data Analysis)

Mục đích của bước phân tích dữ liệu là thu thập thông tin tổng quan về tập dữ liệu, từ đó có những điều chỉnh và lựa chọn phù hợp cho các bước tiếp theo như tiền xử lý, lựa chọn mô hình, và cấu hình tham số huấn luyện.

Các khía cạnh phân tích bao gồm:

3.2.3.1. Thống kê số lượng mẫu:

- Tổng số lượng ảnh cho từng loại bệnh.
- Tỷ lệ phân bố giữa các lớp (các loại bệnh). Việc này giúp đánh giá sự cân bằng của tập dữ liệu, một yếu tố quan trọng ảnh hưởng đến hiệu suất của mô hình.

3.2.3.2. Phân tích kích thước ảnh:

- Thống kê kích thước trung bình, độ lệch chuẩn, kích thước nhỏ nhất và lớn nhất của các ảnh trong tập dữ liệu.
- Việc này giúp xác định kích thước đầu vào phù hợp cho mô hình, đảm bảo giữ lại được nhiều thông tin quan trọng trong ảnh.

3.2.3.3. Phân tích kênh màu:

- Trực quan hóa phân bố giá trị pixel theo từng kênh màu (R, G, B) dưới dạng biểu đồ histogram.
- Tính toán giá trị trung bình và độ lệch chuẩn của từng kênh màu trên toàn bộ tập dữ liệu.
- Phân tích ma trận tương quan giữa các kênh màu. Việc này có thể gợi ý về các phép biến đổi màu sắc phù hợp trong bước tăng cường dữ liệu.

3.2.3.4. Đánh giá độ tương phản và độ sắc nét:

- Tính toán độ tương phản trung bình của các ảnh trong tập dữ liệu sử dụng các phương pháp như phương sai của histogram, phương pháp Michelson, hoặc RMS contrast.
- Đánh giá độ sắc nét của ảnh bằng các phương pháp như Laplacian variance.
- Thông tin này giúp hiểu rõ hơn về chất lượng ảnh và có thể gợi ý về các kỹ thuật tăng cường dữ liệu phù hợp như cân bằng biểu đồ (histogram equalization) hoặc làm sắc nét ảnh (sharpening).

3.2.3.5. Trực quan hóa dữ liệu:

- Hiển thị ngẫu nhiên một số mẫu từ mỗi lớp dữ liệu để có cái nhìn trực quan về đặc điểm của từng loại bệnh.
- Sử dụng các kỹ thuật như t-SNE hoặc UMAP để nhúng dữ liệu từ không gian chiều cao xuống không gian 2D hoặc 3D, giúp trực quan hóa sự phân bố của các lớp dữ liệu và đánh giá mức độ tách biệt giữa chúng.

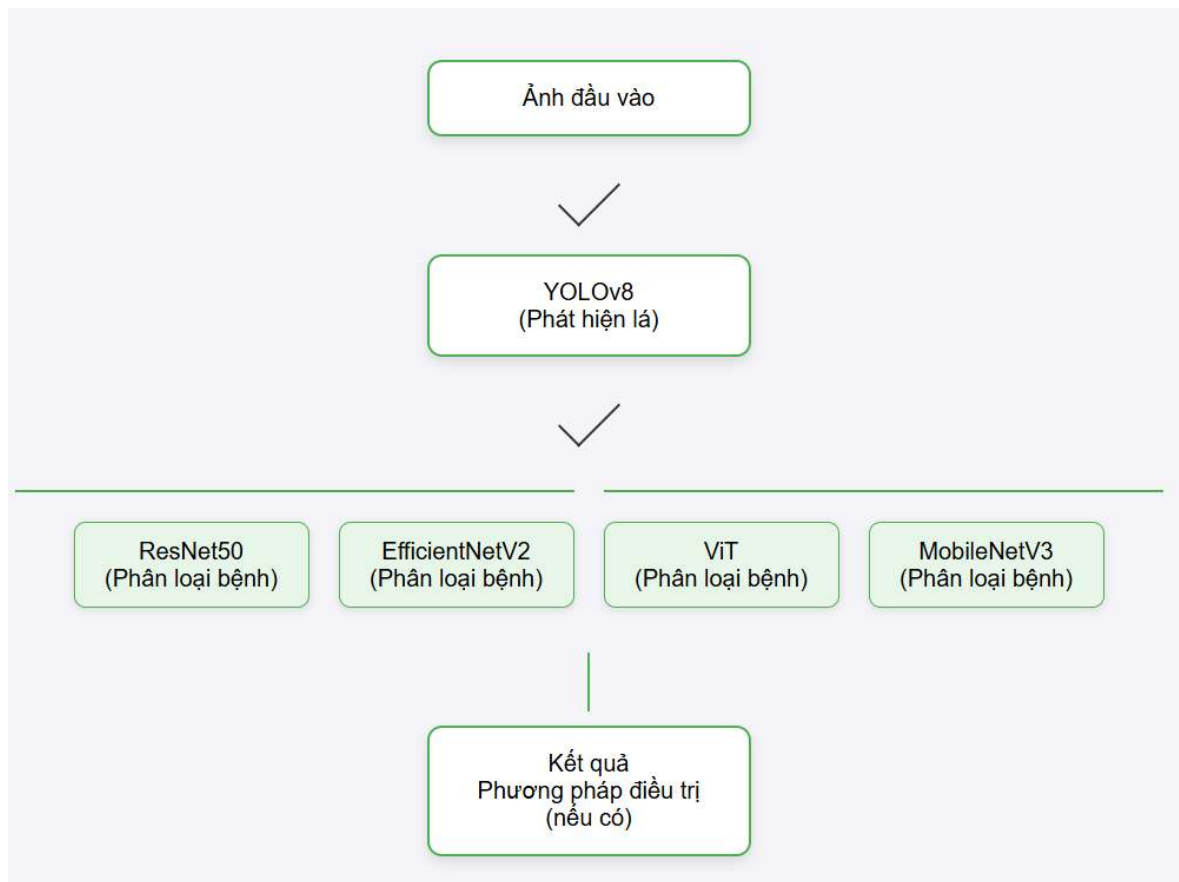
3.3. Thiết kế hệ thống

3.3.1. Kiến trúc tổng thể

Hệ thống nhận dạng sâu bệnh hại cây sản được thiết kế gồm hai mô-đun chính:

1. Mô-đun phát hiện lá sản (Leaf Detection): Sử dụng mô hình YOLOv8 để phát hiện vị trí của các lá sản trong ảnh đầu vào.

2. Mô-đun phân loại bệnh (Disease Classification): Sử dụng các mô hình CNN (ResNet50, EfficientNetV2, ViT, MobileNetV3) để phân loại bệnh.



Hình 3.5: Sơ đồ kiến trúc tổng thể của hệ thống.

3.3.2. Quy trình xử lý

Quy trình xử lý của hệ thống được mô tả như sau:

1. Đầu vào: Ảnh chụp cây sắn hoặc lá sắn.
2. Phát hiện lá sắn: Ảnh đầu vào được đưa qua mô hình YOLOv8 để phát hiện các vùng chứa lá sắn. Kết quả đầu ra là các bounding boxes bao quanh các lá sắn.
3. Tiền xử lý: Các ảnh lá sắn được tiền xử lý (thay đổi kích thước, chuẩn hóa) để phù hợp với đầu vào của mô hình phân loại.
4. Phân loại bệnh: Các ảnh lá sắn sau khi tiền xử lý được đưa qua mô hình phân loại (ResNet50, EfficientNetV2, ViT, MobileNetV3) để dự đoán loại bệnh (đốm lá, cháy lá, khảm lá do virus, thối rễ, lá khỏe mạnh).
5. Đầu ra: Kết quả dự đoán loại bệnh (nếu có) cho từng lá sắn trong ảnh đầu vào và phương pháp chữa trị (nếu có)

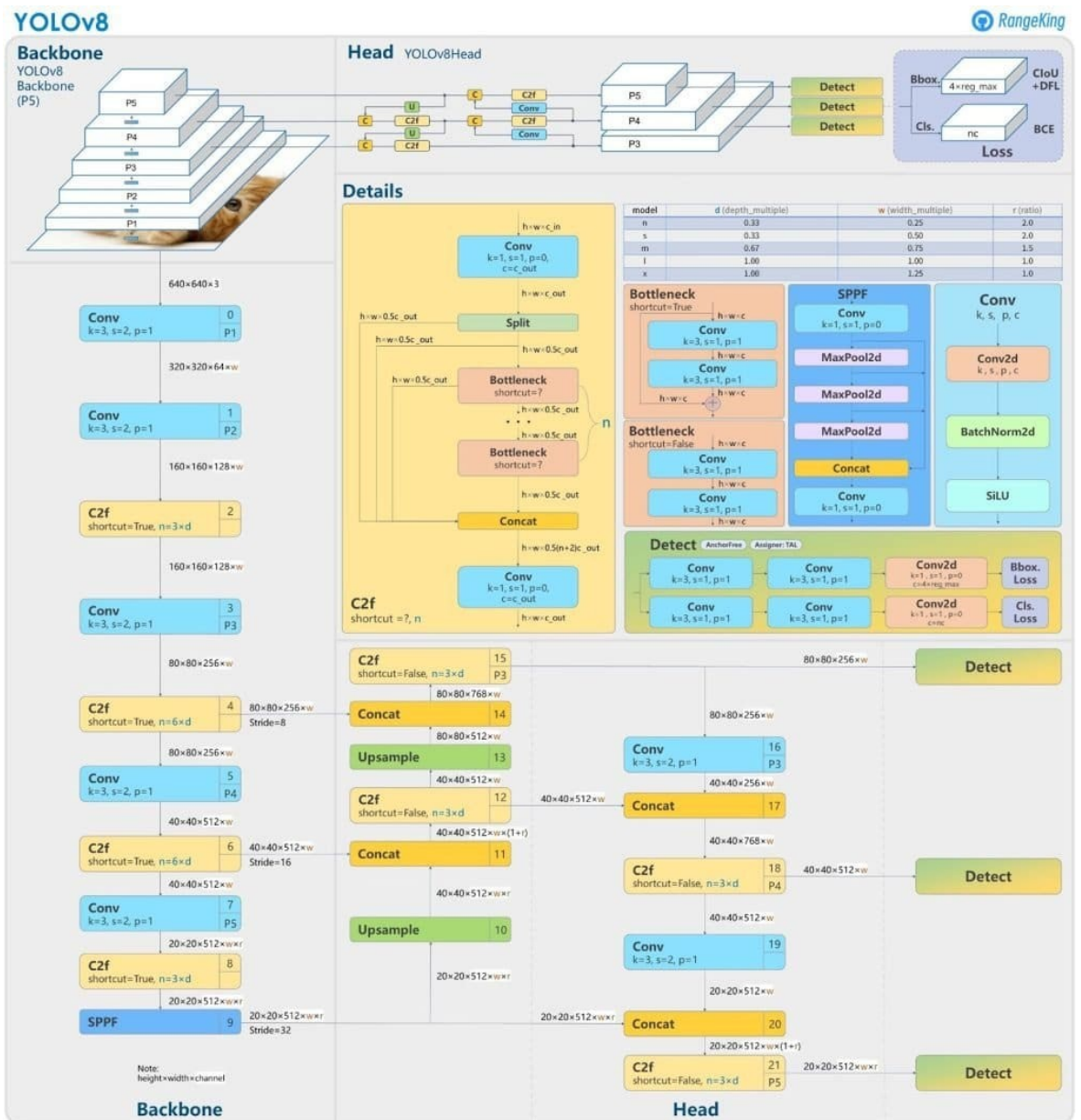
3.3.3. Giải thích các bước triển khai

- Huấn luyện mô hình YOLOv8:
 1. Chuẩn bị dữ liệu: Sử dụng bộ dữ liệu đã được gán nhãn "lá sắn" (mục 3.2.2).
 2. Cấu hình dataset bằng file data.yaml
 3. Huấn luyện: Sử dụng thư viện ultralytics để huấn luyện mô hình YOLOv8 trên tập dữ liệu đã chuẩn bị.
 4. Đánh giá: Đánh giá mô hình trên tập validation để tinh chỉnh tham số và lựa chọn mô hình tốt nhất.
 5. Lưu mô hình: Lưu mô hình đã huấn luyện để sử dụng cho bước phát hiện lá sắn.
- Huấn luyện các mô hình phân loại (ResNet50, EfficientNetV2, ViT, MobileNetV3):
 1. Chuẩn bị dữ liệu: Sử dụng bộ dữ liệu đã được gán nhãn cho bài toán phân loại (mục 3.2.2).
 2. Tiền xử lý: Áp dụng các kỹ thuật tiền xử lý và tăng cường dữ liệu như đã mô tả ở mục 3.2.3.
 3. Load pre-trained models: Sử dụng thư viện timm để tải các mô hình đã được huấn luyện trước trên tập ImageNet (ResNet50, EfficientNetV2, ViT, MobileNetV3) từ thư viện Timm.
 4. Thay thế lớp cuối: Thay thế lớp fully connected cuối cùng của các mô hình bằng một lớp fully connected mới có số đầu ra bằng số loại bệnh cần phân loại (5 loại).
 5. Cấu hình: Cấu hình các tham số cho quá trình huấn luyện (ví dụ: learning rate, batch size, số epoch, optimizer,...).
 6. Huấn luyện: Sử dụng thư viện PyTorch và Timm để huấn luyện các mô hình trên tập dữ liệu đã chuẩn bị.
 7. Đánh giá: Đánh giá các mô hình trên tập validation để tinh chỉnh tham số và lựa chọn mô hình tốt nhất.
 8. Lưu mô hình: Lưu các mô hình đã huấn luyện để sử dụng cho bước phân loại bệnh.

3.4. Mô hình sử dụng

3.4.1. YOLOv8

- Lý do chọn: YOLOv8 là mô hình phát hiện đối tượng (object detection) thuộc họ YOLO, nổi tiếng với tốc độ nhanh và độ chính xác cao. YOLOv8 được lựa chọn cho bài toán phát hiện lá sắn vì:
 - Tốc độ: YOLOv8 có tốc độ xử lý nhanh, phù hợp cho việc triển khai trong thời gian thực.
 - Độ chính xác: YOLOv8 có độ chính xác cao trong việc phát hiện đối tượng.
 - Phù hợp với bài toán: Bài toán phát hiện lá sắn chỉ có một lớp đối tượng duy nhất ("lá sắn"), phù hợp với khả năng của YOLOv8.
- Kiến trúc:



Hình 3.6: Sơ đồ kiến trúc mô hình YOLOv8.

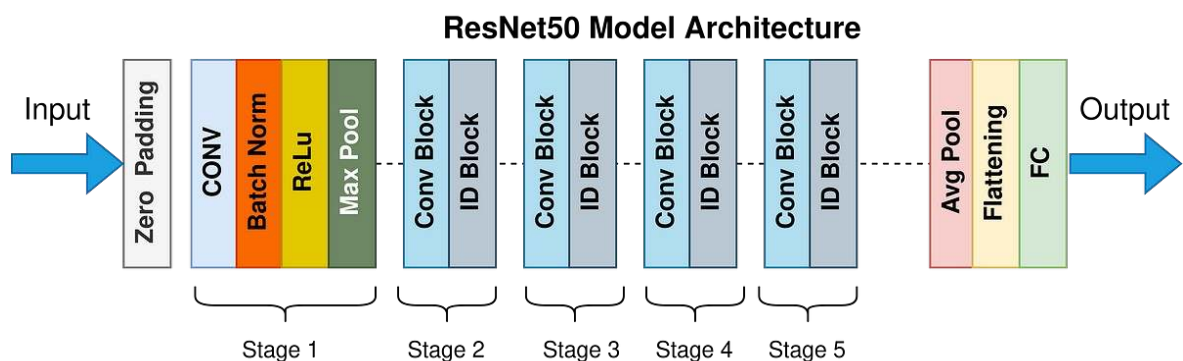
Giải thích: YOLOv8 sử dụng kiến trúc mạng neural network dạng fully convolutional, bao gồm các thành phần chính như backbone, neck và head. Backbone có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào, neck tổng hợp các đặc trưng đó và head đưa ra dự đoán về vị trí và lớp của đối tượng.

- Các hyperparameters:
 - Learning rate: 0.001
 - Batch size: 32
 - Optimizer: Adam
 - Số epoch: 100
 - Input size: 640x640

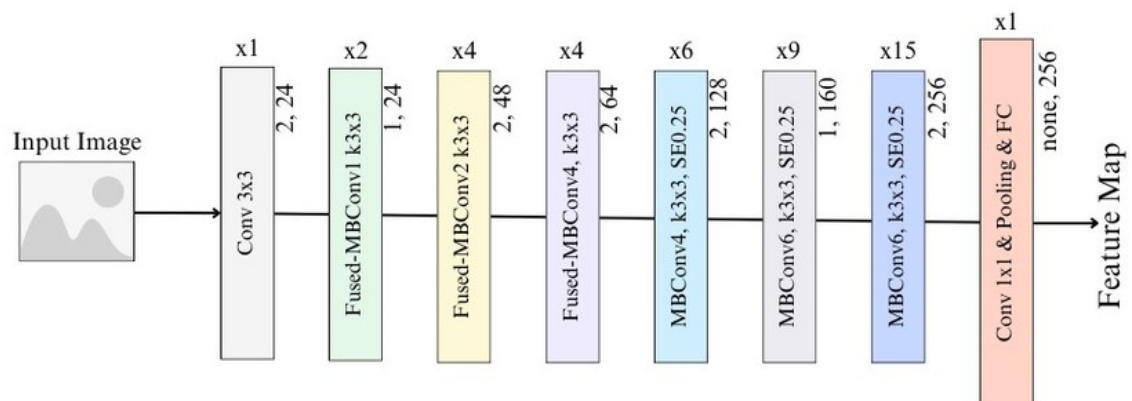
- Các tham số khác: Sử dụng các giá trị mặc định trong thư viện ultralytics.

3.4.2. Các mô hình classification

- Lý do chọn: Các mô hình ResNet50, EfficientNetV2, ViT và MobileNetV3 được lựa chọn để so sánh và đánh giá hiệu quả trong bài toán phân loại bệnh trên lá sầu. Mỗi mô hình có những ưu điểm riêng:
 - ResNet50: Có độ chính xác cao, kiến trúc đã được kiểm chứng qua nhiều nghiên cứu.
 - EfficientNetV2: Hiệu suất tốt, cân bằng giữa độ chính xác và tốc độ suy luận.
 - ViT: Có khả năng học các biểu diễn toàn cục của ảnh, phù hợp với các trường hợp bệnh có đặc trưng phức tạp.
 - MobileNetV3: Kích thước nhỏ gọn, tốc độ suy luận nhanh, phù hợp cho triển khai trên thiết bị di động.
- Kiến trúc:

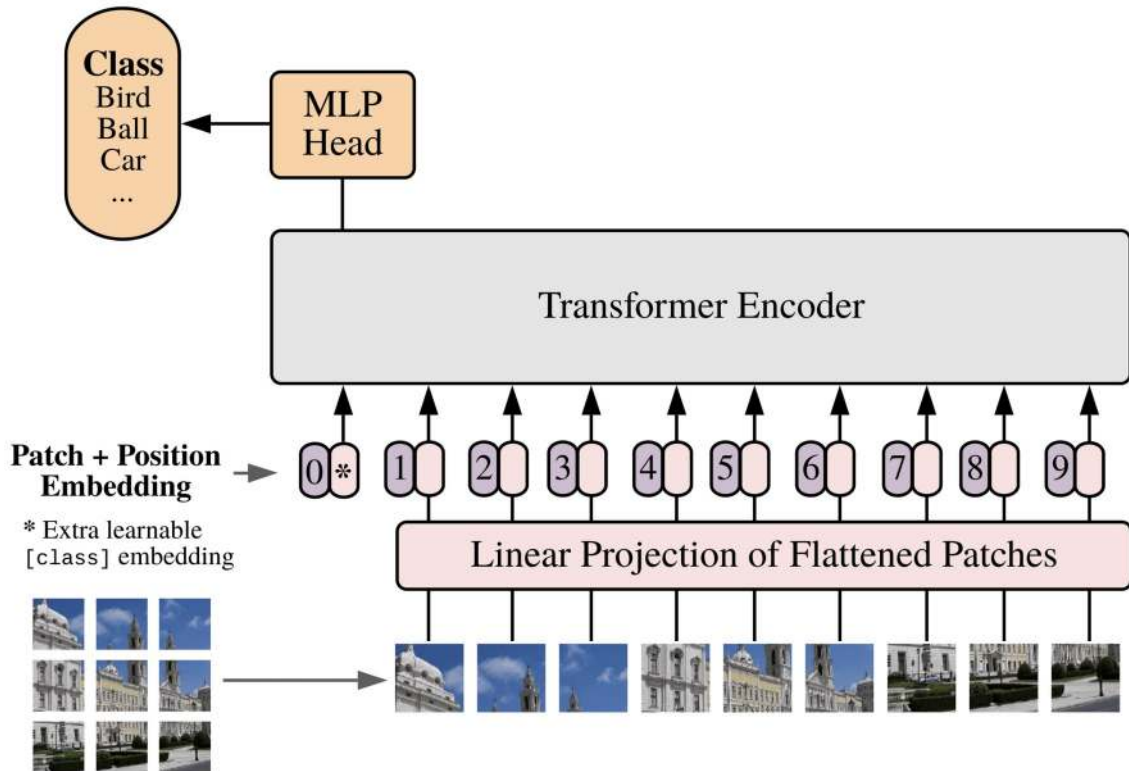


Hình 3.7. Sơ đồ kiến trúc các mô hình ResNet50

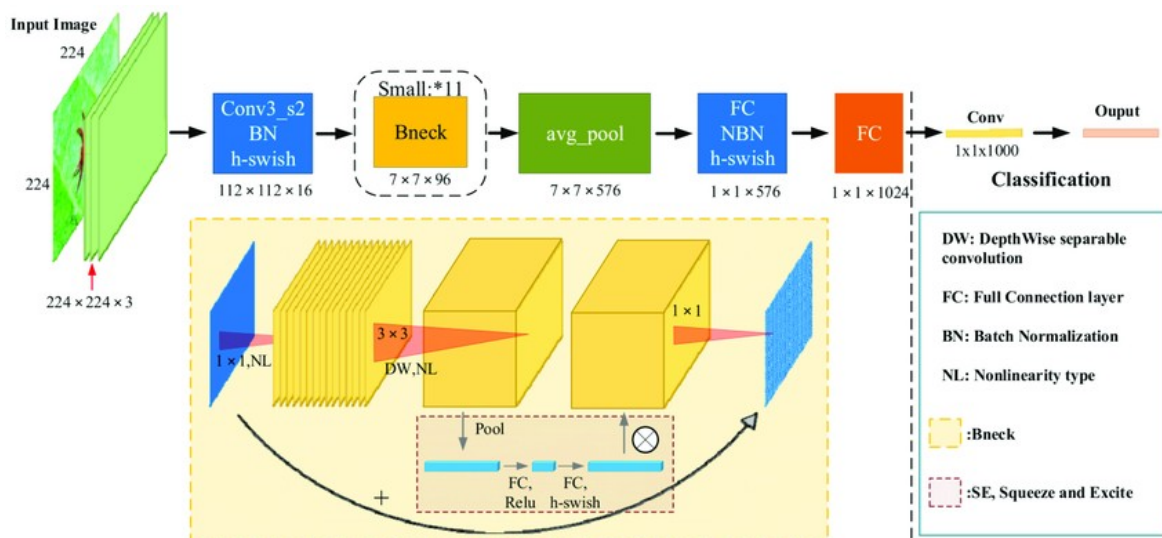


Hình 3.8. Sơ đồ kiến trúc các mô hình EfficientNetV2

Vision Transformer (ViT)



Hình 3.9: Sơ đồ kiến trúc các mô hình ViT



Hình 3.10: Sơ đồ kiến trúc các mô hình MobileNetV3.

a. ResNet50 (Residual Network 50)

- Mô tả: ResNet50 là một mạng neural tích chập (CNN) sâu 50 lớp, nổi tiếng với việc sử dụng các kết nối tắt (residual connections). Những kết nối này cho phép bỏ qua một số lớp, giúp giải quyết vấn đề vanishing gradient trong các mạng neural rất sâu, từ đó cải thiện khả năng huấn luyện và độ chính xác.

- Kiến trúc:
 - Sử dụng các khối "bottleneck" gồm 3 lớp tích chập (1x1, 3x3, 1x1) để giảm chiều sâu và tăng hiệu quả tính toán.
 - Đầu vào được xử lý qua một lớp tích chập và một lớp max-pooling.
 - Tiếp theo là 4 giai đoạn, mỗi giai đoạn chứa nhiều khối residual.
 - Kết thúc bằng một lớp average pooling và một lớp fully connected để phân loại.
- Điểm khác biệt chính: Sử dụng residual connections để cho phép tín hiệu lan truyền trực tiếp qua các lớp, tránh vanishing gradient và cho phép huấn luyện mạng rất sâu.

b. EfficientNetV2

- Mô tả: EfficientNetV2 là một họ các mô hình CNN được phát triển dựa trên Neural Architecture Search (NAS) và compound scaling. Mục tiêu là tối ưu hóa đồng thời độ chính xác và hiệu suất (tốc độ, kích thước).
- Kiến trúc:
 - Sử dụng các khối MBConv (Mobile Inverted Bottleneck Convolution) với Fused-MBConv ở các lớp đầu để tăng tốc độ huấn luyện.
 - Áp dụng compound scaling để mở rộng mô hình một cách cân đối về chiều sâu, chiều rộng và độ phân giải, dựa trên một mô hình cơ sở nhỏ hơn.
 - Sử dụng progressive learning, bắt đầu huấn luyện với ảnh có độ phân giải thấp và tăng dần độ phân giải trong quá trình huấn luyện.
- Điểm khác biệt chính:
 - Tối ưu hóa đồng thời độ chính xác và hiệu suất thông qua NAS và compound scaling.
 - Sử dụng Fused-MBConv để tăng tốc độ huấn luyện.
 - Progressive learning giúp cải thiện độ chính xác và giảm thời gian huấn luyện.

●

c. ViT (Vision Transformer)

- Mô tả: ViT là một mô hình thị giác máy tính dựa trên kiến trúc Transformer, vốn ban đầu được thiết kế cho xử lý ngôn ngữ tự nhiên. ViT phá vỡ truyền thống sử dụng CNN trong thị giác máy tính.
- Kiến trúc:
 - Chia hình ảnh thành các patches cố định (ví dụ: 16x16 pixel).
 - Mỗi patch được biến đổi thành một vector embedding thông qua một phép chiếu tuyến tính.
 - Thêm position embeddings để giữ thông tin vị trí của các patches.
 - Chuỗi các embedding được đưa vào một kiến trúc Transformer tiêu chuẩn, bao gồm các lớp Multi-Head Self-Attention và Feedforward Neural Network.
 - Đầu ra của Transformer được sử dụng để phân loại hình ảnh.
-
- Điểm khác biệt chính:
 - Sử dụng kiến trúc Transformer thay vì CNN.
 - Xử lý hình ảnh như một chuỗi các patches thay vì các pixel riêng lẻ.
 - Hiệu suất tốt khi được huấn luyện trên tập dữ liệu lớn, nhưng cần nhiều dữ liệu hơn CNN để đạt độ chính xác tương đương.
-

d. MobileNetV3

- Mô tả: MobileNetV3 là một họ các mô hình CNN được thiết kế để chạy hiệu quả trên các thiết bị di động và nhúng, với sự cân bằng giữa độ chính xác và tài nguyên tính toán (bộ nhớ, độ trễ).
- Kiến trúc:
 - Kết hợp các cải tiến từ MobileNetV1 (depthwise separable convolutions) và MobileNetV2 (inverted residual blocks).
 - Sử dụng Neural Architecture Search (NAS) để tìm kiếm kiến trúc tối ưu cho từng tác vụ cụ thể.
 - Giới thiệu khối Squeeze-and-Excitation để cải thiện khả năng biểu diễn của mô hình.
 - Sử dụng hàm kích hoạt h-swish để tăng tốc độ tính toán.
-

- Điểm khác biệt chính:
 - Tối ưu hóa cho hiệu suất trên thiết bị di động và nhúng.
 - Kết hợp depthwise separable convolutions và inverted residual blocks.
 - Sử dụng NAS để tự động tìm kiếm kiến trúc tối ưu.
 - Sử dụng khối Squeeze-and-Excitation và hàm kích hoạt h-swish.

Bảng tóm tắt điểm khác biệt chính:

Mô hình	Kiến trúc	Điểm khác biệt chính	Mục tiêu
ResNet50	CNN với Residual Connections	Residual Connections, giải quyết vanishing gradient	Độ chính xác cao cho mạng rất sâu
EfficientNet V2	CNN với NAS và Compound Scaling	Fused-MBConv, Progressive Learning, cân bằng độ chính xác và hiệu suất	Độ chính xác và hiệu suất cao
ViT	Transformer	Không sử dụng CNN, xử lý hình ảnh như chuỗi các patches	Thay thế CNN trong thị giác máy tính
MobileNetV3	CNN với tối ưu hóa cho thiết bị di động	Depthwise separable convolutions, inverted residual blocks, NAS	Hiệu suất cao trên thiết bị di động và nhúng

- Transfer learning: Tất cả các mô hình đều sử dụng pre-trained weights trên tập ImageNet và được fine-tune trên tập dữ liệu bệnh cây sắn.
- Trong nghiên cứu này, tôi đã sử dụng trình tối ưu hóa AdamW để huấn luyện bốn mô hình khác nhau: ResNet50, EfficientNetV2, ViT và MobileNetV3. AdamW là một biến thể của Adam, được cải tiến để xử lý tốt hơn vấn đề suy giảm trọng số (weight decay), giúp ổn định quá trình huấn luyện và thường đạt hiệu suất tốt hơn so với Adam, đặc biệt là với các mô hình Transformer như ViT. Việc lựa chọn AdamW giúp tối ưu hóa quá trình huấn luyện cho cả

các mô hình CNN truyền thống (ResNet50, EfficientNetV2, MobileNetV3) và mô hình dựa trên Transformer (ViT), góp phần cải thiện hiệu suất phân loại bệnh cây sắn.

- Các hyperparameters:
 - Learning rate: 0.001.
 - Batch size: 32
 - Optimizer: AdamW .
 - Số epoch: 25
 - Input size: 224x224
 - Các tham số khác: Sử dụng các giá trị mặc định trong thư viện timm.

CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

Chương này trình bày các kết quả thực nghiệm trong quá trình huấn luyện và thử nghiệm các mô hình, bao gồm đánh giá hiệu suất của mô hình YOLOv8 trong việc phát hiện lá sắn và các mô hình phân loại (ResNet50, EfficientNetV2, ViT, MobileNetV3) trong việc nhận dạng bệnh trên lá sắn. Đồng thời, chương này cũng phân tích các trường hợp lỗi và sử dụng Grad-CAM để trực quan hóa kết quả, từ đó đưa ra những đánh giá và đề xuất cải thiện.

4.1. Quá trình huấn luyện và thử nghiệm

4.1.1. Môi trường thử nghiệm

Các thí nghiệm được thực hiện trên nền tảng Google Colab, sử dụng GPU được cung cấp miễn phí T4. Ngôn ngữ lập trình chính là Python, sử dụng các thư viện PyTorch và Timm cho việc xây dựng và huấn luyện mô hình, OpenCV cho tiền xử lý ảnh, Scikit-learn cho các độ đo đánh giá, Matplotlib và Seaborn để trực quan hóa dữ liệu.

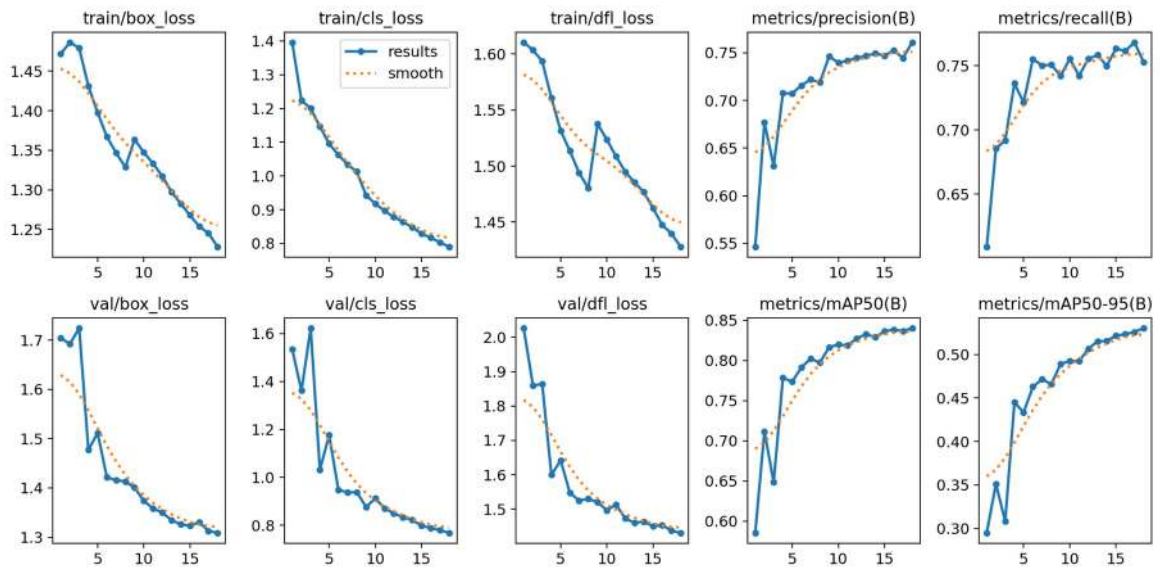
4.1.2. Quá trình huấn luyện và thử nghiệm mô hình YOLOv8

Mô hình YOLOv8s được huấn luyện để phát hiện lá khoai mì trên tập dữ liệu 13.554 ảnh và được đánh giá trên tập dữ liệu hơn gồm 3,853 hình ảnh với 9,869 trường hợp. Dưới đây là các chỉ số đánh giá hiệu suất của mô hình:

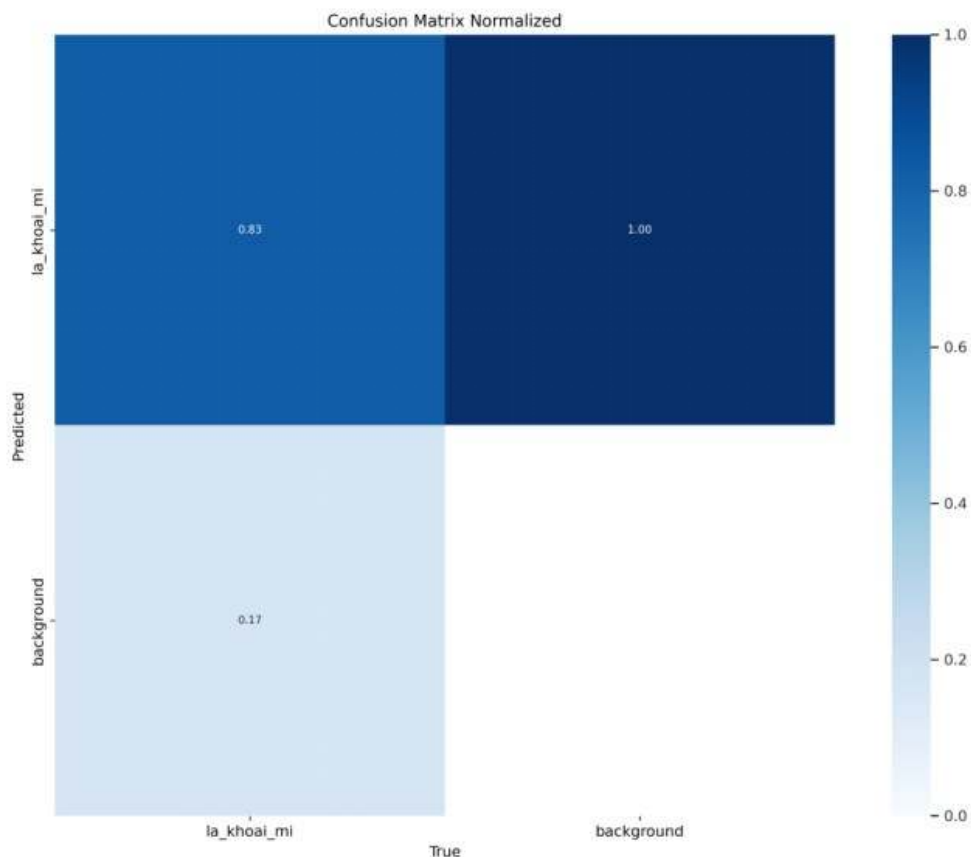
- **mAP@50** (Mean Average Precision tại IoU 50%): Đây là chỉ số chính để đánh giá độ chính xác tổng thể của mô hình trong việc phát hiện đối tượng. Sau 18 epochs, YOLOv8 đạt được mAP@50 là 0.84, thể hiện khả năng cao trong việc phát hiện và định vị đúng lá khoai mì trong ảnh.
- **mAP@50-95**: Độ chính xác trung bình tại nhiều mức giao nhau (IoU) từ 50% đến 95%. mAP@50-95 của mô hình đạt 0.53, cho thấy khả năng mô hình vẫn có thể phát hiện chính xác ngay cả khi đối tượng bị chụp ở nhiều góc độ và kích thước khác nhau.
- **Precision và Recall**:
 - Precision là 0.761, nghĩa là tỷ lệ dự đoán chính xác trên tổng số dự đoán của mô hình. Mô hình không mắc nhiều lỗi dương tính giả (false positives), giúp đảm bảo rằng khi mô hình nhận diện một lá khoai mì, đó là dự đoán có độ tin cậy cao.

- Recall là 0.753, cho thấy mô hình phát hiện đầy đủ các lá khoai mì trong ảnh với tỷ lệ bỏ sót thấp.

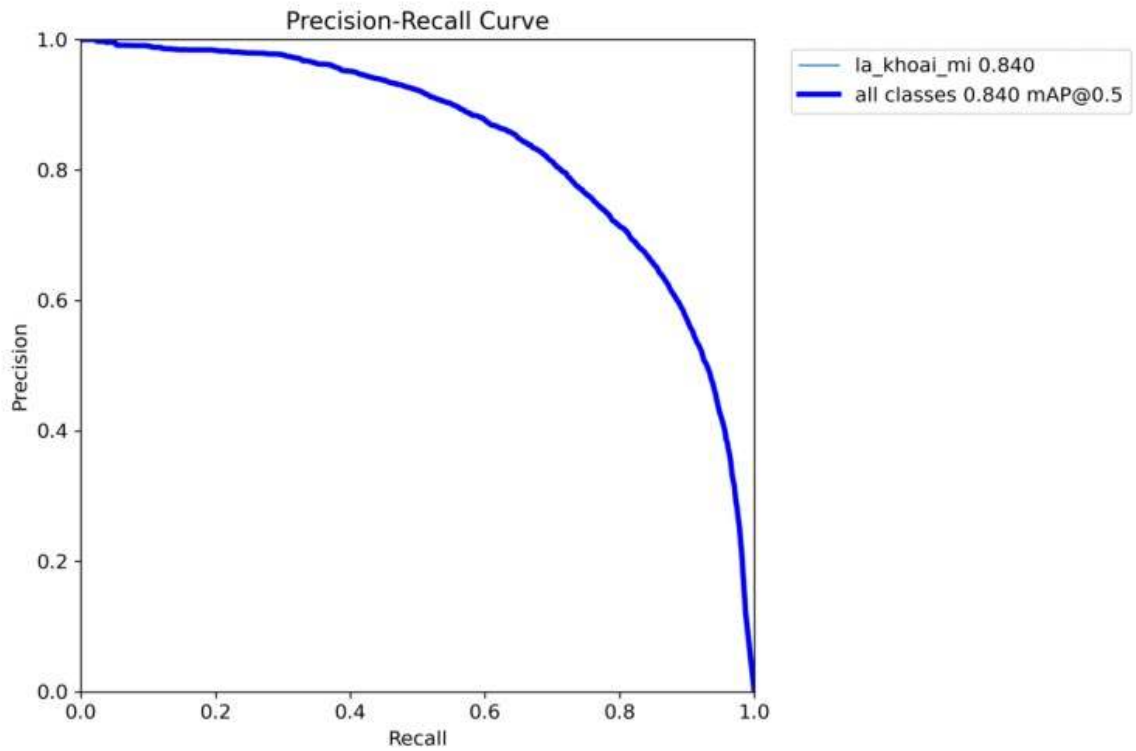
Tốc độ xử lý: Mô hình xử lý nhanh chóng với thời gian inference trung bình là 3.7ms trên mỗi ảnh, đảm bảo khả năng phát hiện lá khoai mì theo thời gian thực, điều này quan trọng khi triển khai trong các hệ thống thực tế.



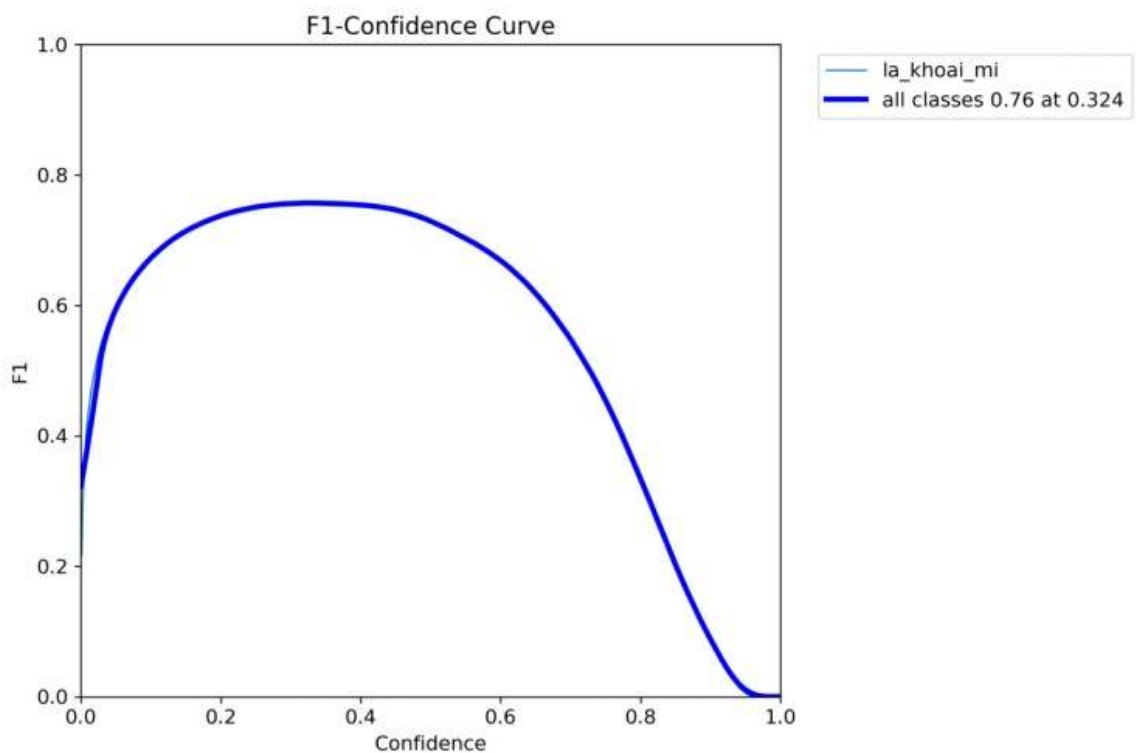
Hình 4.1. Biểu đồ tổng quát kết quả huấn luyện của mô hình



Hình 4.2. Biểu đồ confusion matrix normalized



Hình 4.3. Biểu đồ precision recall



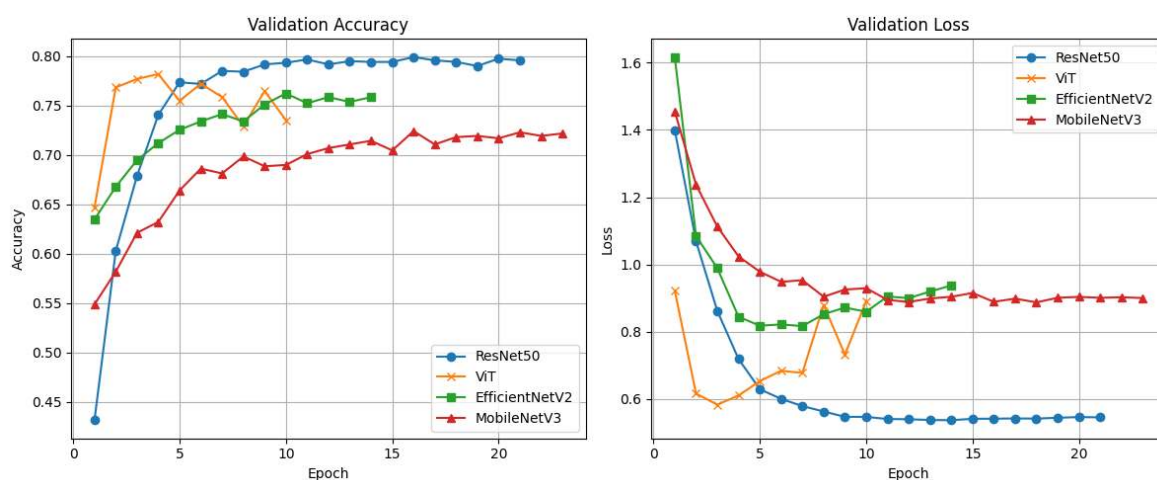
Hình 4.4. Biểu đồ f1 confidence



Hình 4.5. Ảnh batch dự đoán lá khoai mì

4.1.3. Quá trình huấn luyện các mô hình phân loại

Các mô hình phân loại (ResNet50, EfficientNetV2, ViT, MobileNetV3) được huấn luyện trên tập dữ liệu đã được gán nhãn 5 loại bệnh (bao gồm cả lá khỏe mạnh). Quá trình huấn luyện sử dụng các tham số đã được đề cập trong Chương 3 (mục 3.4.2).



Hình 4.6. Biểu đồ val acc và val loss cho bốn mô hình học sâu khi sử dụng các biện pháp tăng cường dữ liệu nhẹ

Nhận xét biểu đồ:

Biểu đồ Validation Accuracy:

- ResNet50: Mô hình ResNet50 cho thấy sự cải thiện độ chính xác ổn định qua các epoch đầu tiên, đạt mức cao nhất khoảng 0.7964 ở epoch 11. Sau đó, độ chính xác có xu hướng dao động nhẹ nhưng vẫn duy trì ở mức khá cao, cho thấy mô hình học tốt và ổn định.
- ViT: Mô hình ViT có độ chính xác ban đầu thấp hơn ResNet50, nhưng cũng cho thấy sự cải thiện đáng kể, đạt đỉnh 0.7816 ở epoch 4. Tuy nhiên, sau đó, mô hình có dấu hiệu overfitting khi độ chính xác giảm và dao động, kết thúc ở mức thấp hơn so với ResNet50.
- EfficientNetV2: EfficientNetV2 khởi đầu với độ chính xác thấp hơn hai mô hình trước, nhưng cho thấy sự cải thiện đều đặn qua các epoch. Mô hình đạt độ chính xác cao nhất 0.7620 ở epoch 10, tuy nhiên độ chính xác có xu hướng giảm nhẹ ở các epoch về sau, có thể do hiện tượng early stopping.
- MobileNetV3: MobileNetV3 có độ chính xác thấp nhất trong số bốn mô hình. Mặc dù có sự cải thiện qua các epoch, nhưng độ chính xác vẫn dao động và đạt mức cao nhất chỉ 0.7239 ở epoch 16, thấp hơn đáng kể so với các mô hình còn lại.

Biểu đồ Validation Loss:

- ResNet50: Mô hình ResNet50 cho thấy sự giảm dần giá trị loss qua các epoch, đạt mức thấp nhất khoảng 0.5369 ở epoch 14, cho thấy mô hình học tốt và hội tụ ổn định.
- ViT: Mô hình ViT có giá trị loss ban đầu khá cao, nhưng giảm nhanh qua các epoch đầu. Tuy nhiên, từ epoch 4 trở đi, loss có xu hướng tăng trở lại, cho thấy dấu hiệu overfitting, tương ứng với sự sụt giảm độ chính xác ở biểu đồ trên.
- EfficientNetV2: EfficientNetV2 có giá trị loss cao ở những epoch đầu, giảm dần và đạt mức thấp nhất 0.8167 ở epoch 7. Tuy nhiên, mô hình bị dừng sớm (early stopping) từ epoch 14, cho thấy mô hình không hội tụ tốt bằng ResNet50.

- MobileNetV3: MobileNetV3 có giá trị loss cao và dao động trong suốt quá trình huấn luyện. Mặc dù có xu hướng giảm, nhưng loss vẫn cao hơn so với các mô hình khác, cho thấy mô hình học kém hiệu quả hơn, tương ứng với độ chính xác thấp hơn.

Nhận xét chung:

- ResNet50 cho thấy hiệu suất tốt nhất về cả độ chính xác và giá trị loss, cho thấy khả năng học và tổng quát hóa tốt trên tập validation.
- ViT có tiềm năng nhưng gặp vấn đề về overfitting.
- EfficientNetV2 cho thấy sự cải thiện nhưng chưa ổn định và bị dừng sớm.
- MobileNetV3 có hiệu suất thấp nhất, cho thấy mô hình không phù hợp với bài toán này khi chỉ áp dụng các biện pháp tăng cường dữ liệu nhẹ.
- Cần lưu ý rằng các mô hình đều sử dụng transfer learning và chỉ áp dụng các biện pháp tăng cường dữ liệu nhẹ. Việc tăng cường dữ liệu mạnh hơn, tinh chỉnh siêu tham số hoặc sử dụng các kỹ thuật tối ưu hóa khác có thể cải thiện hiệu suất của các mô hình.

4.2. So sánh mô hình

Các mô hình được đánh giá dựa trên các độ đo: Accuracy, Precision, Recall, F1-score, và thời gian suy luận (inference time).

Model	Best Val Acc	Best Epoch	Train Acc	Val Loss	Test Acc	F1-score	Inference (ms)	Params (M)	Size (MB)
ResNet50	0.796	11	0.835	0.541	0.800	0.800	22	26.0	98
ViT	0.782	4	0.830	0.611	0.770	0.770	40	86.0	340
EfficientNetV2	0.762	10	0.932	0.859	0.740	0.740	20	30.0	120
MobileNetV3	0.724	16	0.931	0.889	0.710	0.710	8	5.4	22

Bảng 4.6: Bảng so sánh kết quả của các mô hình trên tập test.

Nhận xét biểu đồ :

Biểu đồ Validation Accuracy:

- ResNet50: Mô hình ResNet50 cho thấy sự cải thiện độ chính xác ổn định qua các epoch đầu tiên, đạt mức cao nhất khoảng 0.7964 ở epoch 11. Sau đó, độ chính xác có xu hướng dao động nhẹ nhưng vẫn duy trì ở mức khá cao, cho thấy mô hình học tốt và ổn định.

- ViT: Mô hình ViT có độ chính xác ban đầu thấp hơn ResNet50, nhưng cũng cho thấy sự cải thiện đáng kể, đạt đỉnh 0.7816 ở epoch 4. Đây là một kết quả rất tiềm năng, cho thấy ViT có khả năng đạt được độ chính xác cao nếu được tối ưu hóa thêm. Tuy nhiên, sau epoch 4, mô hình có dấu hiệu overfitting khi độ chính xác giảm và dao động.
- EfficientNetV2: EfficientNetV2 khởi đầu với độ chính xác thấp hơn, nhưng cho thấy sự cải thiện đều đặn qua các epoch. Mô hình đạt độ chính xác cao nhất 0.7620 ở epoch 10. Mặc dù độ chính xác có xu hướng giảm nhẹ ở các epoch về sau, nhưng kết quả này vẫn rất hứa hẹn, cho thấy EfficientNetV2 có tiềm năng phát triển tốt.
- MobileNetV3: MobileNetV3 có độ chính xác thấp nhất trong số bốn mô hình. Mặc dù có sự cải thiện qua các epoch, nhưng độ chính xác vẫn dao động và đạt mức cao nhất chỉ 0.7239 ở epoch 16.

Biểu đồ Validation Loss:

- ResNet50: Mô hình ResNet50 cho thấy sự giảm dần giá trị loss qua các epoch, đạt mức thấp nhất khoảng 0.5405 ở epoch 14, cho thấy mô hình học tốt và hội tụ ổn định.
- ViT: Mô hình ViT có giá trị loss ban đầu khá cao, nhưng giảm nhanh qua các epoch đầu. Tuy nhiên, từ epoch 4 trở đi, loss có xu hướng tăng trở lại, cho thấy dấu hiệu overfitting, tương ứng với sự sụt giảm độ chính xác ở biểu đồ trên. Việc áp dụng các kỹ thuật tối ưu hóa như regularization, điều chỉnh learning rate, hay tăng cường dữ liệu mạnh hơn có thể giúp ViT cải thiện kết quả.
- EfficientNetV2: EfficientNetV2 có giá trị loss cao ở những epoch đầu, giảm dần và đạt mức thấp nhất 0.8167 ở epoch 7. Việc áp dụng early stopping cho thấy mô hình có thể được cải thiện hơn nữa nếu tiếp tục huấn luyện với các điều chỉnh thích hợp.
- MobileNetV3: MobileNetV3 có giá trị loss cao và dao động trong suốt quá trình huấn luyện. Mặc dù có xu hướng giảm, nhưng loss vẫn cao hơn so với các mô hình khác.

Nhận xét chung :

- ResNet50 hiện tại đang cho thấy hiệu suất tốt nhất về cả độ chính xác và giá trị loss.
- ViT và EfficientNetV2 cho thấy tiềm năng phát triển rất lớn. Mặc dù hiện tại chưa đạt được kết quả tốt nhất, nhưng với độ chính xác ban đầu khá cao và xu hướng cải thiện rõ rệt, hai mô hình này hoàn toàn có thể đạt được kết quả tốt hơn ResNet50 nếu được tối ưu hóa thêm. Các kỹ thuật như tăng cường dữ liệu mạnh hơn (MixUp, CutMix, AutoAugment...), tinh chỉnh siêu tham số (learning rate, weight decay, optimizer...), sử dụng các kỹ thuật regularization (dropout, L1/L2 regularization), hay áp dụng các kiến trúc ViT/EfficientNetV2 mới hơn có thể được áp dụng để cải thiện hiệu suất của hai mô hình này.
- MobileNetV3 cho thấy hiệu suất chưa cao ở thời điểm hiện tại. Tuy nhiên, với lợi thế về kích thước nhỏ và tốc độ suy luận nhanh, MobileNetV3 vẫn là một lựa chọn tốt cho các thiết bị có tài nguyên hạn chế như IoT hay mobile. Việc tinh chỉnh mô hình (pruning, quantization), sử dụng kiến thức chưng cất (knowledge distillation) hoặc áp dụng các kiến trúc MobileNetV3 hiệu quả hơn có thể giúp cải thiện độ chính xác của mô hình này.

4.3. Trực quan hóa với XAI

Để hiểu rõ hơn về cách thức hoạt động của mô hình và lý giải nguyên nhân dẫn đến các dự đoán đúng hoặc sai, kỹ thuật Explainable AI (XAI) được áp dụng. Cụ thể, phương pháp Grad-CAM (Gradient-weighted Class Activation Mapping) được sử dụng để tạo ra các bản đồ nhiệt (heatmap) trực quan hóa vùng ảnh mà mô hình tập trung vào khi đưa ra quyết định phân loại.

4.3.1. Giới thiệu về Grad-CAM

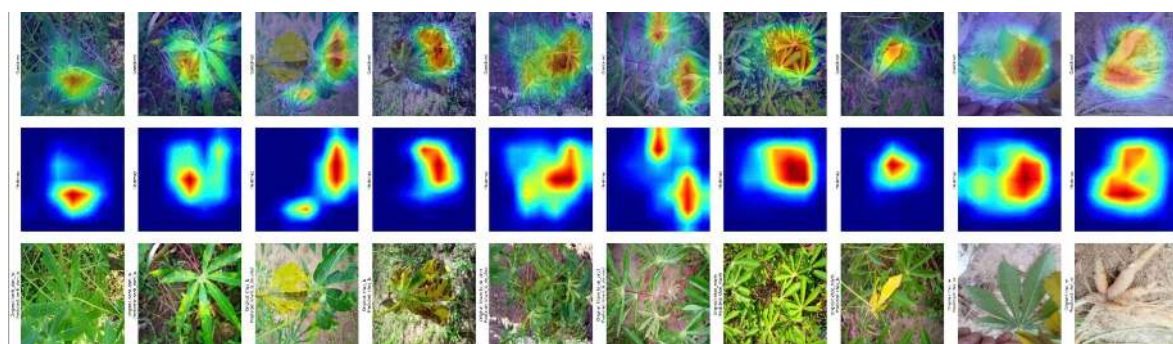
Grad-CAM là một kỹ thuật phổ biến trong XAI, cho phép tạo ra bản đồ nhiệt làm nổi bật các vùng ảnh quan trọng nhất đối với dự đoán của mô hình CNN. Grad-CAM sử dụng gradient của điểm số phân loại (classification score) đối với các feature map của lớp tích chập cuối cùng để tính toán trọng số cho từng feature map. Sau đó, bản đồ nhiệt được tạo ra bằng cách kết hợp tuyến tính các feature map với trọng số tương ứng. Kết quả là một bản đồ nhiệt có độ phân giải thấp, cho biết những vùng ảnh nào có ảnh hưởng lớn nhất đến quyết định phân loại của mô hình.

4.3.2. Ứng dụng Grad-CAM cho ResNet50

Trong nghiên cứu này, Grad-CAM được áp dụng cho mô hình ResNet50 để trực quan hóa quá trình ra quyết định của mô hình khi phân loại các bệnh trên lá sắn. Bằng cách chồng bản đồ nhiệt lên ảnh gốc, ta có thể thấy rõ vùng lá sắn nào (vùng có màu đỏ/vàng trên heatmap) được mô hình ResNet50 chú ý nhất khi phân loại.

4.3.3. Phân tích kết quả Grad-CAM

Dưới đây là một số ví dụ về kết quả trực quan hóa Grad-CAM cho mô hình ResNet50 trên tập dữ liệu test:



Hình 4.7. Kết quả Grad-CAM cho các trường hợp phân loại đúng và sai của mô hình ResNet50

- Trường hợp 1, 2, 6 (bệnh đốm lá): Mô hình phân loại chính xác bệnh đốm lá. Heatmap cho thấy mô hình tập trung vào các vùng lá có đốm nâu, vàng, là đặc trưng của bệnh này.
- Trường hợp 3 (cháy lá): Mô hình dự đoán sai thành khảm lá. Heatmap cho thấy mô hình tập trung vào phần rìa lá bị cháy, tuy nhiên, có thể do phần rìa lá này không có màu nâu đỏ điển hình (có thể do thiếu sáng) nên mô hình bị nhầm lẫn.
- Trường hợp 4 (cháy lá): Mô hình phân loại chính xác bệnh cháy lá. Heatmap tập trung vào phần lớn lá bị cháy, có màu nâu sẫm.
- Trường hợp 5 (khảm lá): Mô hình phân loại chính xác bệnh khảm lá. Heatmap tập trung vào vùng lá có các đốm màu vàng loang lổ và các vùng lá bị xoắn, là triệu chứng điển hình của bệnh.
- Trường hợp 7 (khỏe mạnh): Mô hình phân loại chính xác lá khỏe mạnh. Heatmap cho thấy mô hình tập trung vào toàn bộ lá, không có vùng nào nổi bật, cho thấy lá không có dấu hiệu bệnh.

- Trường hợp 8 (khỏe mạnh): Mô hình phân loại nhầm thành bệnh cháy lá. Heatmap cho thấy mô hình tập trung vào phần bóng của lá, có màu tối hơn. Đây có thể là nguyên nhân khiến mô hình nhầm lẫn với triệu chứng cháy lá.
- Trường hợp 9 (thối rữa): Mô hình phân loại chính xác bệnh thối rữa. Heatmap cho thấy mô hình tập trung vào các củ sắn có dấu hiệu thối rữa (màu nâu sẫm và chảy nước).
- Trường hợp 10 (thối rữa): Mô hình phân loại chính xác bệnh thối rữa. Heatmap cho thấy mô hình tập trung vào củ sắn, đặc biệt là các vùng có màu nâu đậm hơn.

Kết luận:

- Grad-CAM cung cấp một cách trực quan để hiểu được cách thức mô hình ResNet50 đưa ra dự đoán.
- Trong các trường hợp phân loại đúng, heatmap thường tập trung vào các vùng lá có triệu chứng bệnh điển hình.
- Trong các trường hợp phân loại sai, heatmap có thể chỉ ra nguyên nhân sai sót, ví dụ như mô hình tập trung vào các vùng nhiễu (bóng, phần không phải lá) hoặc các triệu chứng không rõ ràng.
- Kết quả Grad-CAM có thể được sử dụng để cải thiện mô hình, ví dụ như bằng cách điều chỉnh trọng số cho các loại bệnh, bổ sung thêm dữ liệu huấn luyện cho các trường hợp khó phân loại, hoặc áp dụng các kỹ thuật tiền xử lý ảnh để làm nổi bật các đặc trưng quan trọng.

Hướng phát triển:

- Trong tương lai, Grad-CAM sẽ được áp dụng cho các mô hình khác (ViT, EfficientNetV2, MobileNetV3) để so sánh và đánh giá khả năng giải thích của từng mô hình.
- Nghiên cứu sử dụng các phương pháp XAI khác (như LIME, SHAP) để có cái nhìn đa chiều hơn về cách thức hoạt động của các mô hình.
- Phân tích chi tiết hơn các trường hợp phân loại sai dựa trên kết quả Grad-CAM để đề xuất các giải pháp cải thiện mô hình hiệu quả hơn.